



SafeSpeech: a three-module pipeline for hate intensity mitigation of social media texts in Indic languages

Koyel Ghosh^{1,2} · Neeraj Kumar Singh¹ · Joy Mahapatra¹ · Saptarshi Saha¹ · Apurbalal Senapati² · Utpal Garain¹

Received: 31 August 2024 / Revised: 9 November 2024 / Accepted: 22 November 2024 / Published online: 12 February 2025
© The Author(s) 2025

Abstract

Warning: This paper contains some abusive text that might sometimes be found offensive.

Identifying and mitigating hateful, abusive, offensive comments on social media is a crucial, paramount task. It's challenging to entirely prevent such hateful content and impose rigorous censorship on social platforms while safeguarding free speech. Recent studies have focused on detecting hate speech, whereas mitigating the intensity of hate remains unexplored or somewhat complex. This paper introduces a cost-effective, straightforward, and novel three-module pipeline, **SafeSpeech**, for Hate Speech Classification, Hate Intensity Identification, and Hate Intensity Mitigation on social media texts. The initial module classifies text as either containing or not containing hate speech. Following this, the second module quantifies the intensity of hate associated with individual words within the classified hate speech. Lastly, the third module seeks to diminish the overall hatefulness conveyed in the text. A comprehensive experiment has been conducted using publicly available datasets in five Indic languages (Hindi, Marathi, Tamil, Telugu, and Bengali). The system undergoes thorough evaluation to assess its performance and analyze it in-depth using various automated metrics. We evaluated the performance of Hate Speech Classification using Precision, Recall, and F1 metrics. For Hate Intensity Identification, we use human evaluation. Recognizing the limitations of automated metrics in mitigating hate speech, we augment our experiments with human evaluation for Hate Intensity Mitigation, where three domain experts independently participated. BERTScore for final generated hate-mitigated texts and first classified hate texts across all languages consistently range between 0.96 and 0.99.

Keywords Hate speech detection · Pre-trained language model · Hate intensity identification · Explainability · Integrated gradients · Hate intensity mitigation

Koyel Ghosh and Neeraj Kumar Singh have contributed equally to this work.

Mr. Singh worked on this project till February 2024.

✉ Koyel Ghosh
ghosh.koyel8@gmail.com
Neeraj Kumar Singh
neeraj1909@gmail.com
Joy Mahapatra
joymahapatra90@gmail.com
Saptarshi Saha
saptarshi.saha_r@isical.ac.in
Apurbalal Senapati
a.senapati@cit.ac.in
Utpal Garain
utpal@isical.ac.in

1 Introduction

Over half of the global population, including print media, is now active on social media, collectively spending 12.5 trillion hours online (Reportal 2021; Kemp 2022, 2024). This surge in social media usage has facilitated the proliferation of hateful content. One such highly hateful content is 'hate speech', broadly defined as any spoken, written, or behavioral communication that targets and discriminates against individuals or groups based on identity traits such as religion, ethnicity, nationality, race, gender, or other characteristics, often employing derogatory language (Fischer et al. 2018; Nations 2023; Waseem and Hovy 2016). Over time, hate speech is expected to rise, yet aggressive posts, misinformation, and hostile comments can all catalyze social unrest, outbreaks of violence, and various revengeful actions: (Laub 2019).

¹ Indian Statistical Institute, Kolkata, India

² Central Institute of Technology Kokrajhar, Kokrajhar, India

1.1 Motivations

Major digital platforms like XTM, FacebookTM, and others are increasingly focused on detecting and filtering hate speech by integrating modules to restrict such language and threats (Hemphill 2023; Yaraghi 2023; O'Regan 2023; Times 2019; Bloomberg 2019) as the United Nations has introduced its action plan (Guterres et al. 2019). Now, addressing hate speech on social media often involves flagging content, deleting comments/ posts, imposing restrictions on user profiles, and suspending users. However, this can push banned users to less regulated platforms (voxp0l 2023) or lead to criticism for restricting free speech (ohchr 2023). Effective detection and mitigation of hate speech could address this issue comprehensively. Twitter and Instagram have introduced experimental prompts that flag offensive posts and allow users to revise them before posting (Hutchinson 2019, 2020). Research by Twitter shows that these prompts lead to fewer offensive tweets compared to users who weren't prompted. This approach offers a more user-friendly alternative to outright bans. While extensive research exists on hate speech detection (Kennedy et al. 2020; Mathew et al. 2020; Cercas Curry et al. 2021; Vidgen et al. 2021), some on hate intensity identification (Masud et al. 2022; Meng et al. 2023) and hate reduction (Masud et al. 2022) for resource-rich languages like English, similar efforts for low-resource Indic languages are lacking, leaving hate mitigation underdeveloped. While in the area of hate intensity identification, prior studies such as by Meng et al. (2023) have relied heavily on manually annotated data, tagging hate intensity for specific words. For hate intensity mitigation, previous research (Masud et al. 2022) has typically required supervised training on annotated datasets to detect hate spans and reduce hate intensity. A key limitation of this approach is its dependence on a hate-tagged corpus, which is costly and challenging to compile.

1.2 Research objectives

Motivated by the above gaps, our goal is to develop alternative solutions to content filtering and user bans before content is made public. Our approach aims to overcome these limitations by introducing an automated method that functions independently of large amounts of labeled data. Deliberate and visionary awareness-raising among online users can empower them to express their views without inadvertently promoting harm. For example, the statement in Bangla, “শালা বাংলাদেশি মা*****” (*English* - “*Shala Bangladeshi mo******”, *Roman* - “*Shala Bangladeshi mo******”) is a hate text, and our system will rewrite the

text as “অবশ্যই বাংলাদেশি একটা” (*English* - “*Definitely a Bangladeshi one*”, *Roman* - “*Aboshyoi Bangladeshi ekta*”) with less intensity of hate (Fig. 1).

This paper demonstrates two different transformer-based BERT (Bidirectional Encoder Representations from Transformers) models (Devlin et al. 2018), i.e., multilingual Google-MuRIL (Khanuja et al. 2021) and monolingual L3Cube-pune (Joshi 2022a, b) models for hate text classification in the social media post, calculate hate intensity of words in the text with an automatic approach unlike (Meng et al. 2023) which needs a part of manually tagged hate intensity for words. Ultimately, use MASK and REPLACE the hate words to mitigate the hate intensity of the text without any further annotation like (Masud et al. 2022). We use Macro-F1 score, Human Evaluation, and BERTScore (Zhang et al. 2019) to evaluate the three-module system.

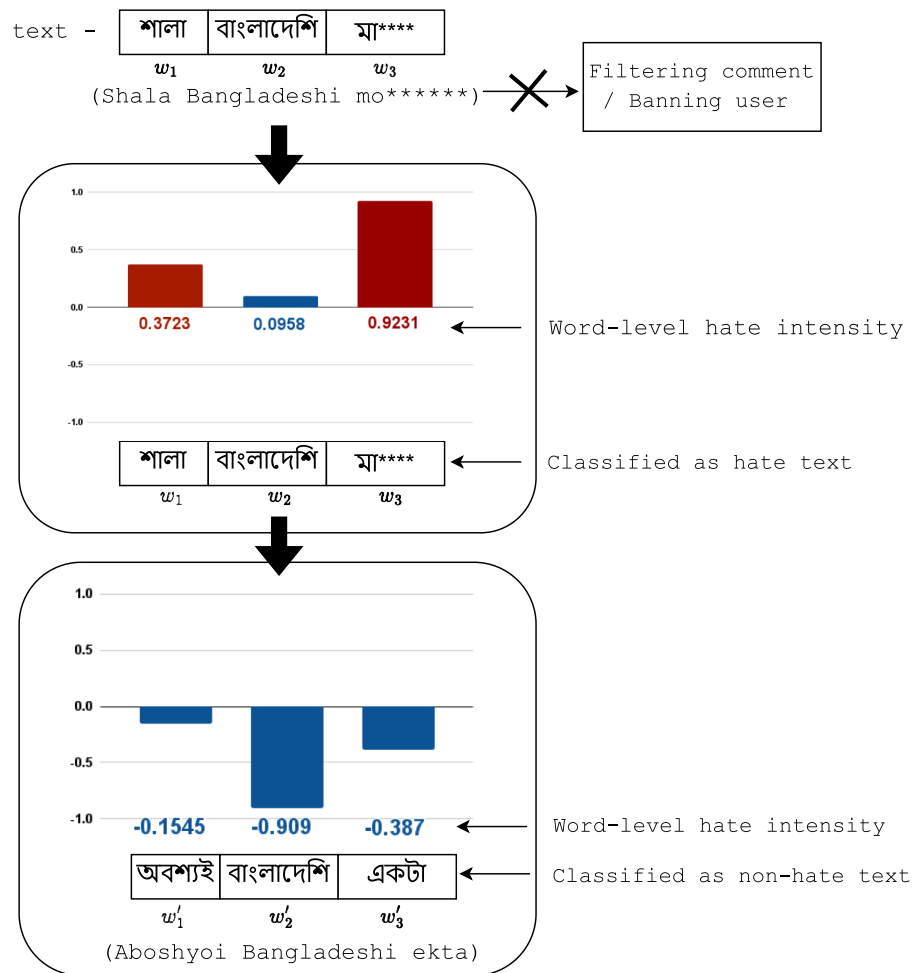
1.3 Contributions

In principle, we are aligned with free speech ethics. As proponents of free speech, we aim to advocate an alternative approach. We focus on mitigating hate intensity rather than promoting hate speech provocation or banning users. Our approach carefully considers the risks of altering the meaning or intent of user-generated content during mitigation. We strive to balance hate speech reduction with respect for free expression, taking steps to avoid unintentionally censoring non-hateful but provocative language. This ensures the mitigation process preserves user intent while contributing to a safer online environment. Our work addresses significant gaps in hate intensity identification and mitigation by introducing an innovative approach that reduces reliance on costly, hard-to-obtain hate-tagged datasets. Unlike traditional methods focused on content filtering and user bans, our solution proactively mitigates hate intensity before content publication. This contribution shifts the paradigm, allowing for automatic, context-aware moderation without the need for extensive labeled data, enhancing both the efficiency and scalability of hate mitigation efforts across diverse contexts.

In summary, the main focus of this paper is below:

1. We propose **SafeSpeech**, an innovative, sustainable, and streamlined system that automatically identifies high-intensity hate words driving a model's predictions. Requiring minimal annotation, **SafeSpeech** mitigates hate speech on social media by replacing these harmful words with benign alternatives.
2. The proposed system is evaluated in different datasets of low-resourced Indian languages to test its performance.
3. A comprehensive evaluation with Macro-F1 score, Human Evaluation, and BERTScore has been done for

Fig. 1 An example demonstrating the **SafeSpeech** system, which is detecting hate texts, identifying the word-level hate intensity, and rewriting the text with mitigated hate intensity. It mitigates the hate text “শালা বাংলাদেশি মা****” (where hate intensity is 0.3723, 0.0958, 0.9231 for each word and classified as hate) to “অবশ্যই বাংলাদেশি একটা” (where hate intensity is -0.1545, -0.909, -0.387 for each word and classified as non-hate)



the system's modules, i.e., $\mathcal{M}_{HSC}$, $\mathcal{M}_{HI}$ and $\mathcal{M}_{HIM}$, respectively.

4. An extensive error analysis has been presented at the end of the paper.

The rest of the paper is structured as follows: Sect. 2 is the existing works on hate speech classification, hate speech classification for Indic language, explainable method, hate intensity identification and mitigation. Section 3 explains the methodology. Section 4 discusses the Experiments, which include datasets, settings, and results. Section 5 discusses error analysis, and Sect. 6 illustrates the conclusion.

2 Related work

In this section, we explore prior research, with a particular focus on hate speech classification (Table 1), hate classification on Indic languages (Table 1), explainable

methods (Table 2), hate intensity calculation, and mitigation (Table 3).

2.1 Hate speech classification

The evolution of hate speech attributes has shifted resolution strategies from traditional template matching to classical machine learning methods and now to advanced deep learning techniques. Hate context detection techniques primarily rely on the use of discriminative classification methods (like text classification) and explainable approaches. In hate speech detection, most of the work is done as text classification. Greevy and Smeaton (2004) takes challenges to identify racism as the mere presence of specific indicator words is not conclusive for recognizing racist texts, unlike certain other text classification tasks. They explore three distinct representations of a web page within the Support Vector Machines (SVM) (Cortes and Vapnik 1995) framework, namely, bag-of-words, bigrams, and part-of-speech tags, considering various interpretations of what constitutes

Table 1 A summary of hate speech classification

References	Language focus	Algorithm
Greevy and Smeaton (2004)	English	SVM
Cortes and Vapnik (1995)	English	SVM
Kwok and Wang (2013)	English	Naive Bayes Classifier
Burnap and Williams (2014)	English	Voted Ensemble
Burnap and Williams (2016)	English	BOW and SVM
Davidson et al. (2017)	English	TF-IDF and Logistic Regression
Greevy and Smeaton (2004)	English	BOW, PoS tagging and SVM
Liu and Forss (2014)	English	N-gram
Waseem and Hovy (2016)	English	N-gram
Mehdad and Tetreault (2016)	English	N-gram
Chen (2011)	English	Lexical Syntactic Feature
Schmidt and Wiegand (2017)	English	SVM, RNN, Bootstrapping
Albadi et al. (2019)	English	N-gram and Gated RNN
Dinakar et al. (2021)	English	Rule-based Jrip, Tee-based J48 and SMO (SVM)
Djuric et al. (2015)	English	BOW and paragraph2vec
Badjatiya et al. (2017)	English	CNN, LSTM and FastText
Nobata et al. (2016)	English	N-gram, Word2Vec and comment2vec
Djuric et al. (2015)	English	BOW and paragraph2vec
Saksesi et al. (2018)	English	RNN
Das et al. (2021)	English	Attention RNN
Anand and Eswari (2019)	English	CNN and LSTM
Paul et al. (2023)	English	CNN, BiLSTM and BERT
Nagar et al. (2023)	English	Variational Graph Auto-encoder
Maity et al. (2024)	English	BERT
Singh et al. (2025)	English	BERT
Bashar and Nayak (2020)	Hindi	Pre-Trained Word Vectors + CNN
Mandl et al. (2019)	Hindi, German and English	N/A
Raj et al. (2020)	English, Hindi, German	Glove+ FastText & CNN+BiLSTM
Mandl et al. (2020)	Hindi, German, English, Tamil and Malayalam	N/A
Modha et al. (2021)	Hindi, Marathi, English	N/A
Bhardwaj et al. (2020)	Hindi	BERT + SVM, MLP, Neural Networks
Ghosh and Senapati (2022)	Hindi, Marathi, and Bangla	BERT, RoBERTa, ALBERT, and DistilBERT + MLP
Romim et al. (2020)	Bangla	Word2Vec, FastText, BengFastText + SVM
Ghosh and Senapati (2024)	Hindi, Marathi, Bangla, Assamese and Bodo	BERT, RoBERTa, ALBERT, and DistilBERT + MLP
Ghosh et al. (2023a)	Assamese and Bodo	Word2Vec and Skip-Gram + SVM, LSTM, CNN and BERT models
Ghosh et al. (2023b)	Assamese	mBERT and Bangla-BERT + mBERT and Bangla-BERT
Sharma et al. (2022)	Hindi-English	MuRIL BERT

discriminatory language. In another study (Kwok and Wang 2013), a supervised machine learning methodology leverages labelled data sourced from various Twitter (now, X) accounts. The data is employed to train a binary classifier, distinguishing between the labels ‘racist’ and ‘nonracist’. Burnap and Williams (2014) introduces a text classifier based on supervised machine learning. The classifier is trained and evaluated to differentiate between responses characterized as hateful or antagonistic, with a specific emphasis on those related to race, ethnicity, or religion,

and more generic responses. In addition to the studies mentioned, another popular method for automatic hate speech detection relies on N-grams (Burnap and Williams 2016; Davidson et al. 2017; Greevy and Smeaton 2004; Liu and Forss 2014; Waseem and Hovy 2016). Assembling lists of N words by connecting sequences of words is the most typical N-gram method. In this scenario, they tally up all the occurrences of expressions of size N. As a result, the performance of classifiers can be enhanced by using this information to understand the meaning of individual words

Table 2 A summary of explainable methods

References	Explanation method/ Model
Yang et al. (2016)	Rule-Based algorithm
Wang et al. (2020)	Hidden Markov Model (HMM)
Nguyen and Martínez (2019)	MonoNet
Li et al. (2018)	Inherent Explanation (using AutoEncoder and ProtoType Layer)
Hase et al. (2019)	Prototype Model
Bahdanau et al. (2014)	Bahdanau Attention
Vaswani et al. (2017)	Transformer Model
Madsen et al. (2022)	Post-hoc Interpretability
Ribeiro et al. (2016)	LIME
Lundberg and Lee (2017)	SHAP
Binder et al. (2016)	LRP
Shrikumar et al. (2017)	DeepLIFT
Springenberg et al. (2014)	Deconvolution
Agarap (2018)	Guided Back-propagation
Noh et al. (2015)	Deconvolution Network
Sundararajan et al. (2017)	Integrated Gradients
Clarke et al. (2023)	Rule-by-Example

Table 3 A summary of hate intensity calculation and mitigation

References	Hate mitigation approach
Masud et al. (2022)	Paraphrasing Hate Spans
Meng et al. (2023)	DRAGNET++
Dahiya et al. (2021)	DESSERT
Davidson et al. (2017)	Crowd-sourced Hate Speech Lexicon
Wiegand et al. (2018)	Domain-independent Lexicon
Hallinan et al. (2023)	MARCO Detoxification

better. N-grams can also be used with characters or syllables in place of words. This method is less affected by occasional changes in spelling than others. Regarding abusive language identification, character N-gram features were more helpful than word N-gram features (Mehdad and Tetreault 2016). However, there are drawbacks to using N-grams. A potential drawback is that even closely related words can be far apart in a text (Burnap and Williams 2016), and solutions to this issue, like increasing the N value, slow down processing performance (Chen 2011). Research also shows that a larger N (≥ 5) improves performance over smaller N values (1 or 3 g) (Liu and Forss 2014). According to a survey (Schmidt and Wiegand 2017), N-gram features are widely regarded as highly predictive in the problem of automatic detection of hate speech. However, they show the most promise when used with other characteristics. Authors Albadi et al. (2019) present the first work in recognition of Arabic tweets using inflammatory and dehumanizing language to incite hatred

and violence based on religious beliefs. The authors created the first public Arabic dataset for religious hate speech detection and three public Arabic lexicons with hate scores. They analyze the dataset, highlighting the most targeted religious groups and the origins of hateful tweets. The dataset is then used to train and evaluate seven classification models, incorporating lexicon-based, n-gram-based, and deep learning approaches to assess their generalization capabilities. TF-IDF (term frequency-inverse document frequency) is also applied to sentiment classification problems (Dinakar et al. 2021). The TF-IDF determines the significance of a word inside a corpus of documents. To account for the fact that some words appear more frequently than others in the corpus, the frequency of the term is offset by the frequency of the word in the corpus, setting it apart from an N-gram bag of words (e.g., stop words). Djuric et al. (2015) presents paragraph2vec technique which can determine whether a user's comment contains abusive or appropriate language and guess the message's main word using FastText (Badjatiya et al. 2017) embeddings. Texts, not individual words, must be categorized, which presents a challenge for hate speech identification (Schmidt and Wiegand 2017). The problem can be solved by averaging the word vectors in the text. This approach, however, only works up to a certain point (Nobata et al. 2016). Some authors suggest comment embeddings get around this issue (Djuric et al. 2015). A recurrent neural network (RNN) is a neural network that uses the results of the previous phase as input for the one being performed now. Saksesi et al. (2018) use RNN to classify hate speech in the text. The Disadvantage of RNN is that the gradient descent problem does not improve accuracy after a certain time for the longer texts. Das et al. (2021) used long short-term memory networks (LSTM). Anand and Eswari (2019) use LSTM and CNN with and without word embedding to classify abusive comments. To address this, the authors Paul et al. (2023) developed a new code-switched dataset from Twitter, annotated with binary labels. They employed machine learning (SVM, Logistic Regression) and deep learning models (MLP, CNN, BiLSTM, BERT) to detect cyberbullying in English-Hindi code-switched text. Their proposed model combines hand-crafted features with sequential and semantic patterns generated by advanced deep neural networks. The authors Nagar et al. (2023) propose a novel hate speech detection method on Twitter that integrates textual, social context, and user language features. Their framework, using a Variational Graph Auto-encoder, jointly learns unified features by combining text content with social context and profile information. Designed for flexibility, the framework allows any text encoder to be incorporated, accommodating emerging language models. The authors Maity et al. (2024) explored whether sentiment information improves hate speech detection (HD) in the Thai language and compared the effectiveness of feature engineering versus multitasking

for sentiment-aware HD. Their findings confirmed that sentiment information enhances HD performance and highlighted the comparative strengths of the two approaches. In multilingual societies like India, where code-switched texts dominate the internet, detecting online bullying is more challenging than in monolingual contexts. The authors Singh et al. (2025) propose a curated high-quality dataset of 12,698 Hindi-English code-mixed YouTube comments and replies for detecting misogynistic attitudes. The dataset supports two tasks: identifying content as optimistic, pessimistic, or neutral and categorizing comments as suggestions, appreciation, criticism, offensive, or none. They applied a range of algorithmic models, including machine learning, deep learning, and transformer-based techniques.

Hate Speech classification in English benefits from extensive resources, including large labeled datasets, pre-trained language models like BERT, and a variety of linguistic tools. Due to the widespread use of English online, researchers have access to ample data, enabling high accuracy in detecting various forms of hate speech, including nuanced expressions like slang, sarcasm, and coded language. In contrast, hate speech classification in low-resource languages faces significant challenges due to limited datasets and fewer available language models. In the next part, we shall discuss the works on Hate Speech Classifications on Indic languages.

2.2 Hate speech classification for Indic language

Bashar and Nayak (2020) used Word2Vec word embedding with Convolutional Neural Network (CNN) to classify hate comments in the Hindi language (HASOC dataset) (Mandl et al. 2019) and scored the highest macro F1 score. Raj et al. (2020) scored the highest macro F1 score utilizing CNN and Bidirectional Long Short-Term Memory (BiLSTM) in a hate speech shared task (Mandl et al. 2020). In another hate speech Hindi shared task (Modha et al. 2021), the best submission was achieved the highest Macro F1 fine-tuning Multilingual-BERT. In the paper Bhardwaj et al. (2020), they used the pre-trained multilingual BERT (m-BERT) model for computing the input embedding on the Hostility Detection Dataset (Hindi) later SVM, random-forest, multilayer neural networks, logistic regression models are used as classifiers. In coarse-grained evaluation, SVM reported the best-weighted F1 score. Sharma et al. (2022) tackle the code switch language i.e., Hindi-English fine-tuning MuRIL language model. Ghosh and Senapati (2022) present a detailed comparison of mono and multilingual transformer models with cross-language evaluation on the Hindi (Mandl et al. 2019), Marathi (Modha et al. 2021) and Bangla (Romim et al. 2020) datasets. Later, the work is extended to two more datasets, Bodo and Assamese (Ghosh and Senapati 2024). Machine learning and deep learning methods are applied to new low-resourced datasets for text

classification on ‘hate’ and ‘non-hate’, i.e., Bodo and Assamese (Ghosh et al. 2023a, b).

For low-resourced languages, multilingual models like mBERT or MuRIL are often used, but they lack the precision of models tailored to specific languages. Additionally, linguistic diversity, dialectal variations, and code-switching make it harder to detect hate speech effectively. Researchers must often rely on methods like data augmentation, transfer learning, or cross-lingual techniques to improve accuracy. Yet, the lack of specialized resources still poses a substantial barrier compared to English-focused hate speech detection systems. Now, explainable methods are crucial in AI and machine learning because they help us understand how a model arrives at its predictions or decisions. This is especially important for sensitive applications like hate speech detection, where the stakes are high, and errors can have serious consequences. It also helps in debugging the model by identifying biases or errors in predictions, which is essential for improving model fairness and ethical standards. So, in the next part, our discussion will be on State-of-the-art explainable methods.

2.3 Explainable method

Efforts to determine the significance of explainability have encountered certain limitations. The majority of explanation methods rely primarily on rule-based models, prototype-based models, and attention-based models. The rule-based models operate through sequences of conditional (such as if ...else) rules. Yang et al. (2016) use a set of rules that yield a probabilistic classifier optimizing over rule lists through a mix of theoretical bounds. Risk scores, commonly utilized in fields like medicine and criminal justice, offer decision-makers a comprehension of results. For instance, in the work Wang et al. (2020), Risk scores are employed to identify ‘at-risk’ patients in real-time scenarios. Nguyen and Martínez (2019) established monotonicity between features and outputs, facilitating the independent analysis of a single feature’s impact on output, irrespective of other features. In recent work on prototype-based models, Li et al. (2018) introduced a novel architecture by adding a special prototype layer in the deep network that naturally explains its reasoning for each prediction. The prototypes are learned during training; the learned network naturally explains each prediction. Hase et al. (2019) utilize hierarchically organized prototypes to classify the objects at every level in a predefined taxonomy, giving distinct explanations at each level in a predefined taxonomy. Attention-based models are first introduced by Bahdanau et al. (2014). They extended the idea of a fixed-length vector in neural machine translation to automatically search for the parts of a source text relevant to predicting a target word. Vaswani et al. (2017) introduced the idea of a self-attention mechanism relating different

positions of a single sequence to compute a representation of the sequence. Several post-hoc-based explanation algorithms (Madsen et al. 2022) have been introduced to explain the black-box nature of deep-learning models. One such technique is LIME (Ribeiro et al. 2016), which can explain any classifier's predictions in an interpretable and faithful manner. Another popular approach is SHAP (Lundberg and Lee 2017), which calculates the Shapley value using multiple observations. The work presents a unified framework for interpreting the predictions. Several feature attribution algorithms use saliency maps, such as Layer-Wise Relevance Propagation (LRP) (Binder et al. 2016), DeepLIFT (Shrikumar et al. 2017), Guided Backpropagation (Springenberg et al. 2014), and many others. LRP allows the decomposition of a deep neural network's prediction computed over a sample, like a text, into a relevance score for input embeddings. DeepLIFT (Deep Learning Important Features) breaks down a network's output prediction on a specific input by backpropagating the contributions of all the neurons in the network to every feature of the input. It compares each neuron's activation to its reference activation and assigns contribution scores according to the difference. Guided Backpropagation combines vanilla backpropagation at ReLUs (Agarap 2018) and DeconvNets (Noh et al. 2015). It visualizes the gradient for the input image to capture pixels detected by the neurons. Sundararajan et al. (2017) study the problem of attributing the prediction of a deep network to the input features. They identified two axioms—sensitivity and implementation invariance, that attribution methods should satisfy. They showed that the most well-known attribution methods do not satisfy these axioms. They used this axiom to design the new attribution method called integrated gradient (IG). Clarke et al. (2023) presents a novel exemplar-based contrastive learning approach, i.e., the Rule By Example (RBE) for learning from logical rules for moderation of textual content.

2.4 Hate intensity identification and mitigation

As time progresses, requirements evolve, shifting from mere hate speech detection to proactive hate mitigation and, in some cases, complete elimination. What once was enough is now replaced by more robust solutions, keeping pace with the changing needs. Now, our discussion will focus on hate intensity mitigation. The authors Meng et al. (2023) employ a weighted sum of two measures (Dahiya et al. 2021) to calculate hate intensity for a hate text, which includes the probability that a reply is hateful, as determined by a state-of-the-art hate speech detection model (Davidson et al. 2017), and the average score of all words in the reply derived from a model-independent hate lexicon comprising 2,895 words, as proposed in Wiegand et al. (2018). To reduce the severity of hate speech on online posts, a novel task, the hate intensity reduction task, is introduced Masud et al. (2022)

propose hate speech normalization to provide users with a stepping stone towards non-hate. They manually created a parallel corpus of hate texts and their counterparts (less hateful). The work introduced a hate speech normalization model with three stages—stage 1 measures the hate intensity of the original post; stage 2 identifies the hate span(s) within it; and stage 3 reduces the hate intensity by paraphrasing the hate spans. Hallinan et al. (2023) present MARCO—an algorithm designed for detoxification. MARCO merges controllable generation and text rewriting techniques by employing a Product of Experts alongside autoencoder language models (LMs). The algorithm utilizes likelihoods derived from a non-toxic language model (expert) and a toxic language model (anti-expert) to identify words suitable for masking and replacement. They estimate the effectiveness of their approach across various datasets, focusing on subtle toxicity and microaggressions. The results demonstrate superior performance compared to baseline methods based on automated metrics. Furthermore, in human evaluations, MARCO's rewrites are favored at a rate 2.1 times higher than alternatives.

Based on our comprehensive literature review, we identified research gaps in Hate Intensity Identification and Hate Intensity Mitigation. In the area of Hate Intensity Identification, prior studies such as by Meng et al. (2023) have relied heavily on manually annotated data, tagging hate intensity for specific words. For Hate Intensity Mitigation, previous research (Masud et al. 2022) has typically required supervised training on annotated datasets to detect hate spans and reduce hate intensity. A key limitation of this approach is its dependence on a hate-tagged corpus, which is costly and challenging to compile. Addressing these gaps, our aim is to develop alternative solutions to content filtering and user bans prior to content publication. Our work seeks to address these limitations by proposing an automatic approach that operates without the need for such extensive labeled data.

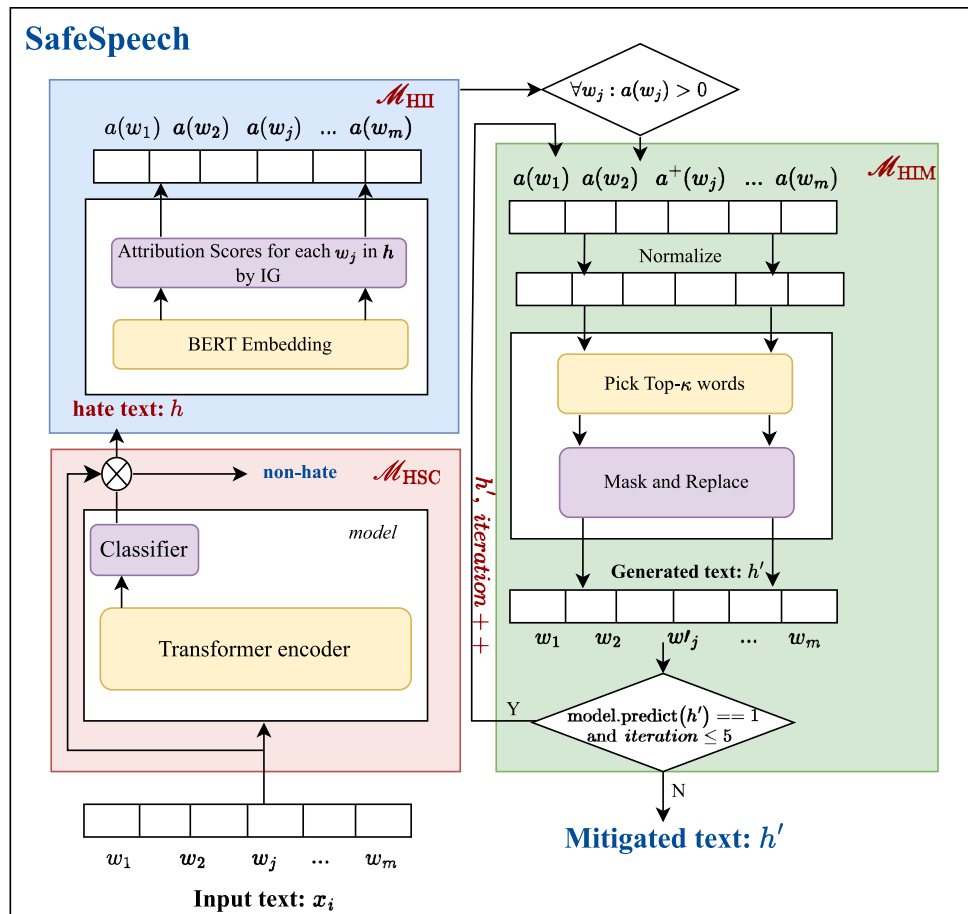
3 Methodology

This section presents our **SafeSpeech** model for hate intensity mitigation of social media texts. Figure 2 provides a high-level overview of the **SafeSpeech** model.

3.1 SafeSpeech

SafeSpeech a three-module pipeline that classifies hate texts, identifies the most hateful words via intensity measurements and mitigates the hate intensity in the texts. Consider an input instance $x \in \mathcal{D}$ of dataset $\mathcal{D}$ consisting of m words. Our objective is threefold. Firstly, we seek to establish whether the input x meets the criteria for hate speech. Secondly, upon identification of hate

Fig. 2 SafeSpeech is a three-module pipeline model. The $\mathcal{M}_{\text{HSC}}$ part classifies hate text, and $\mathcal{M}_{\text{HII}}$ identifies contextually influenced hate words; later, $\mathcal{M}_{\text{HIM}}$ mitigates hatefulness



intensity, we aim to transform the input text (h alias x) into non-hateful text (h'), lastly. To achieve this, three key modules are introduced in the proposed method—(1) Hate Speech Classification ($\mathcal{M}_{\text{HSC}}$), (2) Hate Intensity Identification ($\mathcal{M}_{\text{HII}}$), and (3) Hate Intensity Mitigation ($\mathcal{M}_{\text{HIM}}$). The algorithm 1 shows the steps, which include all three modules.

- **Hate Speech Classification** Initially, it assesses whether the input text (x) qualifies as hate text. Subsequently, upon successful classification of hate text h ($x \rightarrow h$), the next module will be executed; this module is denoted as $h = \mathcal{M}_{\text{HSC}}(x, y)$.
- **Hate Intensity Identification** The module calculates intensity scores h_{int} for each word h_{word} . These scores highlight the words significantly impacting the model's prediction, determining each word's influence on hate speech. We selectively consider only those words h_{word}^+ with positive intensity scores h_{int}^+ , this process is denoted as $h, h_{\text{word}}^+ = \mathcal{M}_{\text{HII}}(h)$.
- **Hate Intensity Mitigation** This module focuses solely on the hate text h and h_{word}^+ . Utilizing the [MASK] and

replace approach, we aim to mitigate the hate intensity present in the text. This process transforms the original hate text h into a mitigated version, denoted as $h' = \mathcal{M}_{\text{HIM}}(x, h, h_{\text{word}}^+)$.

3.2 SafeSpeech-HSC module

The $\mathcal{M}_{\text{HSC}}$ module identifies hate texts through the transformer-based model.

3.2.1 Objective

$\mathcal{M}_{\text{HSC}}$ takes input from a dataset and processes it through a transformer-based model, which generates contextual feature representation. $\mathcal{M}_{\text{HSC}}$ is to determine whether a given text is Hate or Non-hate. The dataset, $\mathcal{D}$, comprises n texts (instances), represented as $\{x_1, x_2, x_3, \dots, x_i, \dots, x_n\}$, where x_i represents the i^{th} text. The text x_i is represented as $x_i = \{w_1, w_2, w_3, \dots, w_j, \dots, w_m\}$, where m is the length, and w_j denotes the j^{th} word of x_i . The dataset can be defined as $\mathcal{D} = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_i, y_i), \dots, (x_n, y_n)\}$, where

each tuple consists of the text (x_i) and its corresponding label (y_i). The label indicates whether the text is Hate or Non-hate. In other words, $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, where y_i is either Hate (1) or Non-hate (0). This task is a binary classification problem, aiming to maximize the objective function in Eq. 1.

$$\arg \max_{\theta} \left(\prod_{i=1}^n P(y_i | x_i; \theta) \right) \quad (1)$$

where x_i is a text with a tagged label y_i to be predicted. θ are the model parameters that require optimization. The process involves creating a classifier to categorize texts into two classes, i.e., training the classifier on the training dataset where the model learns from labelled data, adjusts all parameters to make accurate predictions, and then tests on unseen data to evaluate its performance on the validation dataset to ensure its reliability for real-world applications.

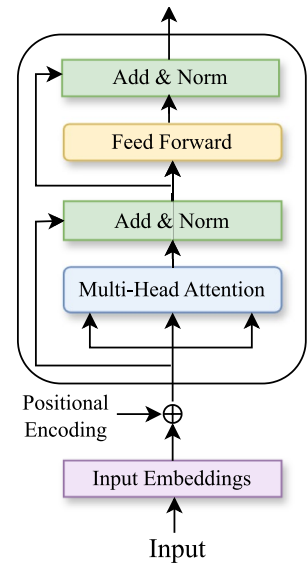
3.2.2 Preprocessing

We applied a few preprocessing steps to clean the dataset and improve its quality for better performance before passing the data to the transformer model for fine-tuning. The transformer model does not require extensive preprocessing, so we perform elementary preprocessing on the text. This includes removing any hyperlinks or links that start with 'www', 'http', or 'https', mentions that begin with '@', new-line characters ('\n'), and most punctuation marks (except full stops ('.'), commas(','), and question marks('?')). If necessary, we also convert class labels to unique numerical values like Hate $\rightarrow 1$ and Non-hate $\rightarrow 0$. In each pair (x_i, y_i) , the text x_i is split into individual tokens based on wordpiece (Sennrich et al. 2015). During tokenization, we incorporate [CLS] (stands for classification) and [SEP] (stands for separator) tokens at the beginning and end of each input text, respectively.

3.2.3 Input representation

Transformer-based models take input (x_i) with token embedding (tok_i), positional embedding (pos_i), and segment embedding (seg_i) for each token which includes [CLS] and [SEP]. Token embedding can transform tokens into a vector representation of a fixed 768 dimension by querying a token embedding matrix, i.e. x_i is embedded as tok_i . Positional embeddings contain information about the position of tokens in sequence, i.e., pos_i , segment embedding stores the text number represented in a vector i.e., seg_i , where the

Fig. 3 Transformer encoder layer



tokens are words or subwords. Final Embedding e_i (2) used by model architecture are the sum of token embedding, positional embedding as well as segment embedding. So, the final embedding will be

$$e_i = tok_i + pos_i + seg_i \quad (2)$$

The final embeddings are subsequently inputted into deep bidirectional layers to generate output. This output comprises hidden state vectors, each with a predetermined size, corresponding to every token in the input sequence. Figure 3 illustrates the Transformer encoder layer, where *Multi-Head Attention* consists of multiple attention mechanisms operating in parallel, each referred to as a head. The *Feed Forward* component is a position-wise neural network. The *Add & Norm* operation represents a residual connection followed by normalization applied after the feedforward network.

3.2.4 Architecture

Our study employs two types of pre-trained transformer-based language models. *Multilingual model*: Google-MuRIL (Khanuja et al. 2021) is a BERT model pre-trained on the dataset of 17 Indian languages and their transliterated counterparts, including monolingual and parallel segments. *Monolingual model*: l3cube-pune/hindi-bert-v2 (Joshi 2022a), l3cube-pune/marathi-roberta Joshi (2022b), l3cube-pune/assamese-bert (Joshi 2022a), l3cube-pune/tamil-bert (Joshi 2022a), l3cube-pune/telugu-bert (Joshi 2022a) and l3cube-pune/bengali-bert (Joshi 2022a) are used to see monolingual performance on the datasets. These models are trained on a particular language dataset.

3.2.5 Fine-tuning

The embedding representation e_i is fed to the pre-trained models. BERT in Eq. 3 consists of 12 transformer layers, and all the layers perform multi-head self-attention operations as well as position-wise full feedforward network linked to calculate the contextual embeddings on e_i , where o_i is the output of the BERT encoder.

$$o_i = \text{BERT}(e_i) \quad (3)$$

We extract the embedding for the [CLS] token from the contextual embedding e_i ([CLS]). After encoding the input sequence with BERT, in the Eq. 4, the special token [CLS] is used to represent the entire sequence. The output corresponding to the [CLS] token is linearly transformed using a linear layer (referred to as *Linear* here). The *softmax* function is then applied to obtain probabilities across different classes. In Eq. 5, if the value of $\hat{y}_i$ (represents the predicted class for the input sequence which determines the index of the class with the highest probability) is equal to 1, then $x_i \in h$ and the second subtask will be executed; otherwise x_i is a not hate text.

$$p_i = \text{softmax}(\text{Linear}(o_i[\text{CLS}])) \quad (4)$$

$$\hat{y}_i = \text{argmax}(p_i) \quad (5)$$

The cross-entropy loss is calculated between the predicted probability distribution and the true label distribution. To minimize the loss function, we train the network. The loss function 6 is shown below:

$$\text{Loss} = -\frac{1}{n} \sum_{i=1}^n \sum_{z=1}^c y_{i,z} \log(p_{i,z}) \quad (6)$$

where n is the total number of instances in $\mathcal{D}$, c is the number of classes, i.e., two for our task. For the sample i , the true label $y_{i,z}$ is 1 if i belongs to class c , otherwise 0. $p_{i,z}$ is the predicted probability and the sample i belongs to class c . Backpropagation computes the loss gradients. The *Adam* optimization algorithm helps to update parameters throughout the training for multiple epochs to minimize the loss. We save the trained *model* and consider the hate text h for further experimentation. HATE_TRAINING function in the algorithm 1 (line no. 11) resembles this subtask.

3.3 Hate intensity identification (HII)

3.3.1 Objective

This module is designed to compute the word-level hate intensity, denoted as h_{int} . It begins by utilizing the predicted hate text h to derive attribution scores h_{atrb} for each hate word in h_{word} .

These attribution scores h_{atrb} are then normalized, producing the final hate intensity measure, h_{int} . We employ a post-hoc approach in our experimental framework to achieve this objective. The first target is calculating the attribution scores for each token (h_{tok}) of h . IGs leverage two key inputs: (a) the input text (for us, it's h) and (b) the baseline text ($\tilde{h}$). The baseline text is carefully chosen to ensure that the model's output is neutral at that point. For the input h and the baseline input $\tilde{h}$, we assess the contribution of each token embedding to the model's predicted class probability. This allows us to determine the attribution for each component of the embedding vector and compute the overall attribution of a token by averaging these values. The formula used to calculate the IGs is given by Eq. 7,

$$IG_{j,k}(h) = (w_{j,k} - \tilde{w}_{j,k}) \times \int_{\alpha=0}^1 \frac{\partial F(\tilde{h} + \alpha \times (h - \tilde{h}))}{\partial w_{j,k}} d\alpha \quad (7)$$

where α is a scalar in the range $[0, 1]$, and $\frac{\partial F(h)}{\partial w_{j,k}}$ is the gradient of $F(h)$ (represents a deep network) along the k^{th} token, here $w_{j,k} \in w_j$. The attribution scores h_{atrb} then normalized to it as hate intensity h_{int} for words h_{word} .

3.3.2 Attribution scores calculation

After fine-tuning and predicting hate texts, we detect the context words that influence a model to classify a text as hate. To attribute the prediction of the model, we use IGs. We tokenized h following the rules of the workpiece and added [CLS], [SEP] for h and set $\tilde{h}$ as the size of h with zeros. $\tilde{h}$ serves as a reference input, a starting point to calculate the attribution of each feature to the model's prediction. In this case, we use a sequence of padding tokens as the baseline. Get the embeddings of the h and $\tilde{h}$, then generate a series of text inputs that gradually move from the $\tilde{h}$ to the h . For each intermediate text input, we calculate the gradient of each layer concerning the preceding one, moving from the classifier layer to the input layer in sequence. We used the chain rule to determine the gradient $\frac{\partial F(h)}{\partial w_{j,k}}$ of the model's predicted class probability for the input text. We then summed these gradients across the entire path from the baseline to the input text, yielding an attribution value for each entry in the embedding vector. To determine the attribution for the tokens of the input text, we averaged the attributions related to their embeddings. The Eq. 8 defines the task correctly as the integral of integrated gradients can be efficiently approximated via a summation.

$$IG_{j,k}(h) = (w_{j,k} - \tilde{w}_{j,k}) \times \sum_{l=1}^s \frac{\partial F(\tilde{h} + \frac{l}{s} \times (h - \tilde{h}))}{\partial w_{j,k}} \times \frac{1}{s} \quad (8)$$

where s is the number of steps (for our tasks it is 50), and $l = 1, \dots, s$. So, $\alpha = [0, \frac{1}{s}, \frac{2}{s}, \dots, 1]$. For each token, the attribution score is calculated, i.e., $a(w_{j,k}) \in h_{atrb}$.

3.3.3 Word level attribution scores

As the sub-word tokens have no semantic sense, we are summing the attribution score of the sub-word tokens to calculate the attribution scores for each word w_j in the hate text h , i.e., $h_{atrb} = \{a([\text{CLS}]), a(w_1), a(w_2), \dots, a(w_j), \dots, a(w_m), a([\text{SEP}])\}$, where $a(w_j)$ is the attribution score of the word w_j of the hate text h .

In all the attribution scores $h_{atrb} = (h_{atrb}^- \cup h_{atrb}^0 \cup h_{atrb}^+)$ and $h_{atrb} \in \mathbb{R}^{m+2}$, positive attribution values (h_{atrb}^+) for features indicate those features contributed to the model's predicted class, while negative values (h_{atrb}^-) suggest features acted against it. A zero attribution (h_{atrb}^0) value implies the features didn't significantly influence the model's prediction for that particular input.

3.3.4 Normalizing attribution scores

However, word level attribution score may not correlate with the overall semantic (hate or non-hate) of the sentence. For example, $a(w_j)$ could be positive while the sentence is classified as negative (non-hate). To solve this issue, we modify the word attribution score in Eq. 9 as follows:

$$a'(w_j) = a(w_j) \wedge \hat{y}_i \quad (9)$$

where $\hat{y}_i$ is the class label of the sentence h and $\wedge$ is the AND operation. It corrects the word level attribution score in the following sense: A positively attributed word (hate) in a non-hate (classified by the model) sentence should be treated with non-hate semantics while a negatively attributed word (non-hate) should remain non-hate always. The case, when the model predicts a sentence as hate text but attributes every word in the sentence as non-hate, will not arrive as IG satisfies completeness axiom (Sundararajan et al. 2017) and our baseline is zero. Again for each hate text from h , we normalize the resultant score ($a'(w_j)$) of the word w_j within that text as follows in Eq. 10:

$$a'(w_j) \rightarrow \frac{a'(w_j)}{\sqrt{\sum_{j=1}^m a'(w_j)^2}} := \tilde{a}(w_j) \quad (10)$$

This forces the attribution vector $\mathcal{A}(h) = (\tilde{a}(w_j))_{j=1}^m$ for an entire text h to lie on the unit $(m-1)$ -sphere S^{m-1} , where m represents the number of words in the text. Now that the attribution vectors for each text with a word length of m lie on S^{m-1} , we aim to define overall hate intensity (using the unit sphere) to compare two hate texts. The overall hate

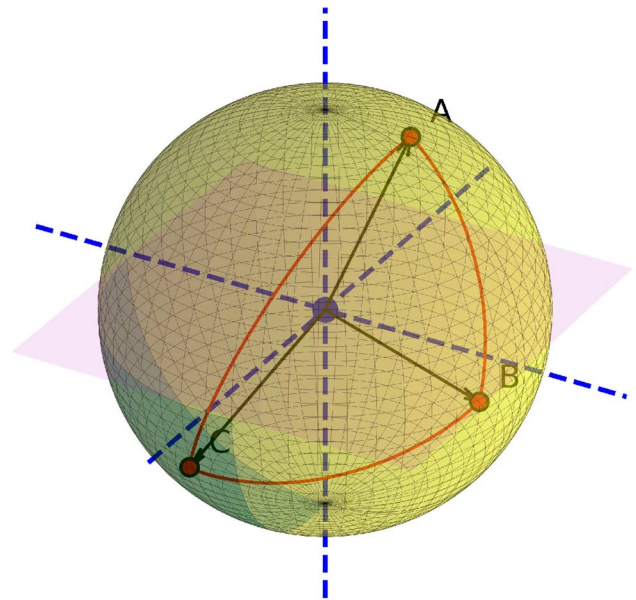


Fig. 4 This is a geometric interpretation of normalized attribution scores of three hate texts: A [0.3723, 0.0958, 0.9231], B [0.9489, -0.3140, -0.0277], C [-0.1545, -0.9090, -0.3870]. Here, A, B and C are - “শালা বাংলাদেশি মা*****”, hate text with low hate intensity (1st iteration) - “শালা বাংলাদেশি একটা” and final non-hate text (last iteration) - “অবশ্যই বাংলাদেশি একটা”. Note that, the green-shaded region of the sphere represents the area where the texts contain no hate words

intensity of a text h can be defined in Eq. 11 i.e., relative to a reference point h^0 ($-\frac{1}{\sqrt{3}}, -\frac{1}{\sqrt{3}}, -\frac{1}{\sqrt{3}}$) of the same word length as the spherical distance between texts:

$$I(h) = \arccos(\mathcal{A}(h) \cdot \mathcal{A}(h^0)) \quad (11)$$

Similarly, we can measure hate intensity differences between three texts. Figure 4 shows the geometrical interpretation of normalized attribution scores and an example of intensity differences between three texts i.e., A - “শালা বাংলাদেশি মা*****” (English - “Shala Bangladeshi mo*****”, Roman - “Shala Bangladeshi mo*****”), B - “শালা বাংলাদেশি একটা” (English - “Shala Bangladeshi one”, Roman - “Shala Bangladeshi akta”) and C - “অবশ্যই বাংলাদেশি একটা” (“Definitely a Bangladeshi one”, Roman - “Aboshyoi Bangladeshi ekta”).

This part gives h and hate words (h_{wrd}^+) which have normalized positive attribution scores, i.e., positive hate intensity (h_{int}^+) with indices w.r.t h_{wrd}^+ as output. The NORM_ATTRIBUTION function (algorithm 1: line no.27) works based on the above theory.

3.4 Hate intensity mitigation (HIM)

Besides detecting hate text and hate words, mitigating hate intensity is also one of our key objectives. In the proposed **SafeSpeech**, the $\mathcal{M}_{\text{HIM}}$ module is primarily responsible for this hate intensity mitigation task.

3.4.1 Objective

$\mathcal{M}_{\text{HIM}}$ takes x, h, h_{word}^+ as input and incorporates a self-supervised learning technique, i.e., MLM, on x . The aim behind using MLM is to calculate the likelihood (Eq. 12) and understand the context of tokens t_a and their relationships with other tokens in a given input $x = \{t_a\}_{a=1}^q$,

$$p(t_a \mid [\text{CLS}], t_1, \dots, t_{a-1}, [\text{MASK}], t_{a+1}, \dots, t_q, [\text{SEP}]) \quad (12)$$

where $[\text{MASK}]$ is a masked token over the a^{th} word. The trained mlm_model will be used to fulfill the mitigation approach i.e., replacement of the $[\text{MASK}]$ words by generating h' (the normalized form of h) whose sentiment y_i will be changed to 0 by reducing the intensity in likelihood (Eq. 13).

$$p(y_i = 0 \mid [\text{CLS}], w_1, w_2, \dots, \text{replace}, \dots, w_m, [\text{SEP}]) \quad (13)$$

3.4.2 Preprocessing

To perform MLM training, we meticulously clean the dataset $\mathcal{D}$. We remove all the punctuation, URLs, and user names. Furthermore, we replace '@' with no space, convert letters to lowercase, and fill in the missing values. Similar tokenization is performed as Sect. 3.2 with truncation and padding $[\text{PAD}]$ up to a maximum length of 512.

Through MLM objective, texts $\{x_1 \cup x_2 \cup \dots \cup x_n\} \in \mathcal{D}$ will be employed to the pre-trained MLM models, during fine-tuning 15% of input texts are randomly masked ($[\text{MASK}]$) except $[\text{CLS}]$, $[\text{SEP}]$, or $[\text{PAD}]$. Here, the models are used the same as mentioned in Sect. 3.2.

3.4.3 Top κ selection and mask-replace words

Top κ words (0.30 of h_{word}^+) are taken, temporarily '[MASK]' those words of the h as well as added $[\text{CLS}]$ at the beginning and $[\text{SEP}]$ at the end of the text. The selected candidates are masked with $[\text{MASK}]$ token which returns h_{mask} . h_{mask} was then employed in the trained mlm_model to calculate the Eq. 14.

$$p(w_j \mid h_{\text{mask}}) \quad (14)$$

where j^{th} word $\in h_{\text{mask}}$ is a masked. MLM is to predict potential alternatives for the masked words. We generate each masked word's top 10 potential alternative words w_{rplc} . The top 10 alternative normalized texts are created from w_{rplc} , i.e., h_{norm} . In the algorithm, step 13 to 14 step explains the above. Out of h_{norm} texts, only the best BERTScore valued (Zhang et al. 2019) text is considered to be the most promising substitute, denoted as h' , in Eq. 15.

$$h' = \text{argmax}(F_{\text{BERTScore}}(h_{\text{norm}}, h)) \quad (15)$$

3.4.4 Iterative mitigation

We follow the iterative approach to check whether the generated h' becomes non-hate text by mitigating its intensity, such as passing the h' to saved classification *model* of $\mathcal{M}_{\text{HSC}}$ for the prediction to check Eq. 5. If the $\hat{y}_i$ becomes zero, then h' is normalized; else, repetition of step Attribution Scores Calculation of Sect. 3.3 to Top κ selection and mask-replace words of Sect. 3.4 until *prediction* of hate text becomes zero ($\text{iteration} \leq 5$). For example, consider Fig. 4, where A (hate text), B (hate text after the 1st iteration), and C (non-hate text after the 2nd iteration) represent three texts processed through iterative refinement, with the number of *iterations* = 2. Here, we found the desired non-hate text from hate text in just two *iterations* only.

Table 4 Class-wise distribution analysis of instances for HASOC, MACD, and BD-SHS datasets

Dataset	Hate			Non-hate			Total
	Train	Val	Test	Train	Val	Test	
HASOC-Hindi	2,469	—	605	2,196	—	713	5,983
HASOC-Marathi	669	—	207	1,205	—	418	2499
MACD-Hindi	10,527	3464	3496	9656	3264	3232	33,639
MACD-Tamil	8302	2824	2795	9698	3176	3205	30,000
MACD-Telugu	9310	3099	3194	8690	2901	2806	30,000
BD-SHS-Bengali	19,324	2416	2416	20,900	2612	2613	50,281

Algorithm 1 SafeSpeech

Input: A set of samples $D = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_i, y_i), \dots, (x_n, y_n)\}$, where each tuple consists of the text (x_i) and corresponding labels (y_i) .

Output: Safe text h' .

```

procedure SAFE_SPEECH( $x, y$ )
     $h \leftarrow \text{HSC}(x, y)$  ▷ Call HSC.
     $h, h_{wrd}^+ \leftarrow \text{HII}(h)$  ▷ Call HII.
     $h' \leftarrow \text{HIM}(x, h, h_{wrd}^+)$  ▷ Call HIM.
    return  $h'$ 
end procedure

procedure HSC( $x, y$ ) ▷ Texts ( $x$ ) and labels ( $y$ ).
     $h \leftarrow \text{HATE\_TRAINING}(x, y)$  ▷ Call Hate_Training.
    return  $h$ 
end procedure

procedure HATE_TRAINING( $x, y$ ) ▷ preprocessing function.
     $x, y \leftarrow \text{Preprocess}(x, y)$ 
     $o_i \leftarrow \text{BERT}(x_i)$ 
     $p_i \leftarrow \text{softmax}(\text{Linear}(o_i[\text{CLS}]))$ 
     $\hat{y}_i \leftarrow \text{argmax}(p_i)$ 
    if  $\hat{y}_i = 1$  then ▷ If classified as hate text  $h \in x$ .
        set  $x$  as  $h$  and save the model
        return  $h$ 
    else
        “Non-hate text detected”
    end if
end procedure

procedure HII( $h$ ) ▷ Hate texts  $h$ 
     $h, h_{wrd}^+ \leftarrow \text{NORM\_ATTRIBUTION}(h)$  ▷ Call Normalized Attribution.
    return  $h, h_{wrd}^+$ 
end procedure

procedure NORM_ATTRIBUTION( $h$ )
    Set  $s, \text{sum} \leftarrow 0, \tilde{h}$  as the sizeof( $h$ ) ▷ Initialize number of steps, the sum of gradients, baseline text.
     $h_{\text{embed}} \leftarrow \text{BERT-embeddings}(h)$ 
     $\tilde{h}_{\text{embed}} \leftarrow \text{BERT-embeddings}(\tilde{h})$ 
     $\alpha \leftarrow [0, \frac{1}{s}, \frac{2}{s}, \dots, 1]$ 
    for  $\alpha_i \in \alpha$  do ▷ Create interpolated embeddings between  $\tilde{h}$  and  $h$ .
         $\text{int\_embed} \leftarrow \tilde{h}_{\text{embed}} + \alpha_i \cdot (h_{\text{embed}} - \tilde{h}_{\text{embed}})$ 
         $\nabla \text{int\_embed} \leftarrow \text{Gradient of int\_embed w.r.t. } p$ 
         $\text{sum} \leftarrow \text{sum} + \nabla \text{int\_embed}$ 
    end for
     $IG(h) \leftarrow \frac{\text{sum}}{s}$  ▷ Embedding Attribution.
     $a(w_{j,k}) \leftarrow \text{attribution}(IG(h))$  ▷ Token Attribution.
     $a(w_j) \leftarrow \text{Aggregate}(a(w_{j,k}))$  ▷ Word Attribution.
     $h_{\text{atrb}}^+ \leftarrow \{w_j : a(w_j) > 0\}$  ▷ Select the words with positive attribution scores.
     $\tilde{a}(w_j) \leftarrow \text{Normalize}(a(w_j))$  ▷ Normalize Attribution scores.
     $h_{wrd}^+, h_{\text{int}}^+ \leftarrow h_{\text{atrb}}^+$  ▷ Normalized positive attribution scores (positive hate intensity),
    i.e.,  $h_{\text{int}}^+ \in \tilde{a}(w_j)$ 
    return  $h, h_{wrd}^+$  ▷ Return the hate text, hate words with the positive hate intensity
end procedure

procedure HIM( $x, h, h_{wrd}^+$ )
     $x \leftarrow \text{Preprocess}(x)$  ▷ Preprocessing same as HSC and removal of punctuations.
     $\text{mlm\_training}(x)$ , save the model as  $\text{mlm\_model}$ . ▷  $\text{mlm\_model}$  includes the
    predictions  $\forall w_j$ .
     $h' \leftarrow \text{MASK\_REPLACE}(h, h_{wrd}^+)$  ▷ Call Mask_Replace.
    while  $\text{model.predict}(h') == 1$  and  $\text{iteration} \leq 5$  do
         $h, h_{wrd}^+ \leftarrow \text{NORM\_ATTRIBUTION}(h)$  ▷ Call Normalized Attribution.
         $h' \leftarrow \text{MASK\_REPLACE}(h, h_{wrd}^+)$  ▷ Call Mask_Replace.
         $\text{iteration} \leftarrow \text{iteration} + 1$ 
    end while
    return  $h'$ 
end procedure

procedure MASK_REPLACE( $h, h_{wrd}^+$ )
     $h_{\text{mask}} \leftarrow \text{Mask}(w_j, h)$ , where  $w_j \in \text{top-k-pick}(h_{wrd}^+, \kappa)$  ▷ [MASK] the top  $\kappa$  hate words
     $h_{\text{norm}} \leftarrow \text{Replace} \begin{cases} w_{\text{rplc}} \leftarrow \text{mlm\_model}_{\text{top10}}(h_{\text{mask}}) & \text{if } w_j \text{ is masked} \\ w_j & \text{otherwise} \end{cases}$  for each  $w_j \in$ 
    ▷ Replace the [MASK] hate words with new words.
     $h' \leftarrow \text{argmax}(F_{\text{BERTScore}}(h_{\text{norm}}, h))$ 
    return  $h'$ 
end procedure

```


4 Experiments

This section includes all the experiments on six benchmark datasets, i.e., two HASOC, three MACD, and BD-SHS, the experiment settings, and the results of $\mathcal{M}_{HSC}$, $\mathcal{M}_{HII}$, and $\mathcal{M}_{HIM}$.

4.1 Datasets

Our experiments involve datasets in five Indian languages: Hindi, Marathi, Tamil, Telugu, and Bengali, for both hate detection and mitigation. To ensure wide coverage, we incorporate three state-of-the-art datasets: the HASOC (Hate Speech and Offensive Content Identification) dataset (Mandl et al. 2019), MACD (Gupta et al. 2024), and BD-SHS (Romim et al. 2022). The class distribution analysis for instances in these six datasets is provided in Table 4.

4.1.1 HASOC

HASOC-Hindi (2019) (Mandl et al. 2019), and HASOC-Marathi (2021) (Modha et al. 2021) is sampled from X^{TM} and FacebookTM using hashtags and keywords. The underlying task of these two dataset is to classify the tweets into two classes (Sub-task A is a coarse-grained binary classification): hate and offensive (HOF) and non-hate (NOT). HOF indicates that a post contains hate speech, offensive language, or both. NOT implies no hate speech or other offensive material in this post. Training and test data are provided separately.

4.1.2 MACD

MACD-Hindi, MACD-Tamil and MACD-Telugu datasets (Gupta et al. 2024) are released by ShareChat in collaboration with CNERG Lab, IIT Kharagpur. All these datasets

are well-balanced and human-annotated, and comments have been sourced from a popular social media platform: Share-Chat. MACD contains the training, validation, and test split in CSV format for all the languages, including Hindi, Tamil, Telugu, Malayalam, and Kannada. The dataset contains two labels - 0 (for abusive comments) and 1 (for non-abusive comments). We Used the Hindi, Tamil, and Telugu datasets of MACD for our experiments.

4.1.3 BD-SHS

BD-SHS-Bengali dataset (Romim et al. 2022) is created by collecting the Bengali article from various sources, including a Bengali Wikipedia dump, Bengali news articles like Daily Prothom Alo, Anandbazar Patrika, BBC, news dumps of TV channels (ETV Bangla, ZEE news), social media (X^{TM} , FacebookTM pages and groups, LinkedInTM), books, and blogs. The raw text corpus consists of 250 million articles. This dataset consists of 30,000 instances, where 10,000 instances belong to the hate category, and 20,000 instances belong to non-hate. Hates are further classified as political, personal, gender abusive, geopolitical, or religious hate.

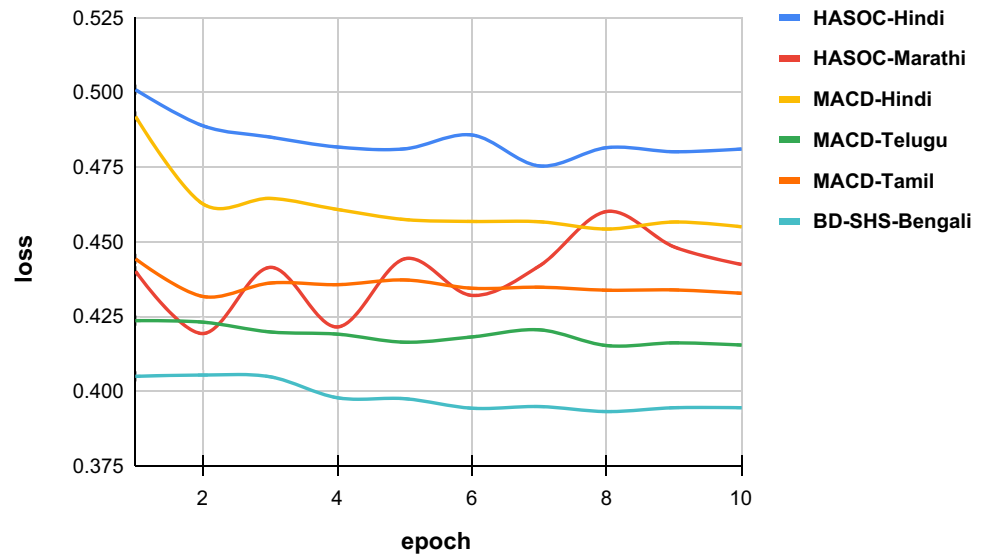
4.2 Settings

We have reported all hate detection results after 5-fold cross-validation. At the training time, early stopping is used for the stopping criteria (Patience= 10). All hyper-parameters are tuned on the validation partition of each dataset. All pre-trained models are downloaded from [Hugging Face](#) (Wolf et al. 2020).

1. For the classification of $\mathcal{M}_{HSC}$ training, AdamW optimizer is used with Learning rate $1e-5$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and momentum = 0.9. In the learning rate scheduler, the warm-up step is 0, the number of training steps = epoch * length (training data), and the loss function is

Table 5 Precision, Recall, and F1 scores for $\mathcal{M}_{HSC}$ on all six datasets

Dataset	Model	Accuracy	Precision	Recall	Macro F1
HASOC-Hindi	google/muril-base-cased	0.84 ± 0.010	0.84 ± 0.010	0.84 ± 0.010	0.84 ± 0.010
	l3cube-pune/hindi-bert-v2	0.84 ± 0.001	0.84 ± 0.001	0.84 ± 0.001	0.84 ± 0.001
HASOC-Marathi	google/muril-base-cased	0.88 ± 0.012	0.87 ± 0.012	0.87 ± 0.010	0.87 ± 0.012
	l3cube-pune/marathi-bert	0.91 ± 0.011	0.90 ± 0.011	0.89 ± 0.011	0.90 ± 0.011
MACD-Hindi	google/muril-base-cased	0.86 ± 0.002	0.86 ± 0.003	0.86 ± 0.003	0.86 ± 0.003
	l3cube-pune/hindi-bert-v2	0.86 ± 0.002	0.86 ± 0.003	0.86 ± 0.003	0.86 ± 0.003
MACD-Tamil	google/muril-base-cased	0.88 ± 0.004	0.88 ± 0.005	0.88 ± 0.005	0.88 ± 0.005
	l3cube-pune/tamil-bert	0.88 ± 0.001	0.88 ± 0.001	0.88 ± 0.001	0.88 ± 0.001
MACD-Telugu	google/muril-base-cased	0.89 ± 0.003	0.89 ± 0.005	0.89 ± 0.005	0.89 ± 0.005
	l3cube-pune/telugu-bert	0.90 ± 0.001	0.89 ± 0.001	0.90 ± 0.001	0.90 ± 0.001
BD-SHS-Bengali	google/muril-base-cased	0.92 ± 0.001	0.92 ± 0.001	0.92 ± 0.001	0.92 ± 0.001
	l3cube-pune/bengali-bert	0.92 ± 0.001	0.92 ± 0.001	0.92 ± 0.001	0.92 ± 0.001

Fig. 5 Changes observed in fine-tuning (Loss Vs Epoch)

Cross-Entropy. The model is trained for 10 epochs with batch size 8 and the dropout rate = 0.2.

- The MLM of $\mathcal{M}_{HIM}$ is trained for 10 epochs with batch size 16. The AdamW optimizer with Learning rate $5e-5$.
- In HSC, we experiment with one multilingual and one monolingual pre-trained BERT model for each language; whichever model gives the best result, we take that model and use that particular model in HIM for MASK and REPLACE too. We choose the same best-performed pre-trained models for classification and MLM training per dataset. Such as Google/mural-base-cased for both the modules HSC and HIM on Hindi datasets, whereas
- 13cube-Pune/Marathi-bert for the Marathi dataset. Likewise, 13cube-pune/telugu-bert for Telugu, 13cube-pune/tamil-bert for Tamil, 13cube-pune/bengali-bert for Bengali. The bold-marked models corresponding to the datasets in Table 5 are used for both of the cases.
- BERTScore metric is used that evaluates text quality using contextual embeddings from BERT.
- The selected hyperparameters were chosen based on a combination of empirical testing and prior knowledge from similar studies. For instance, the learning rate was set to optimize convergence speed without risking instability, while batch size was chosen to balance memory

Table 6 Jaccard index on model-predicted words and human-detected words

Dataset	Sample text	Model-predicted words	Human-detected words	Jaccard index
HASOC (Hindi)	सब रं* नाच देखने मे व्यस्त जैसे ही कोई शांतीदूत के साथ कुछ होगा सब रं*रोना शुरू कर देंगे	रं*, रं*रोना, शांतीदूत	रं*, रं*रोना	0.67
English translation	All are busy watching the wh***'s dance as soon as something happens with the peacekeeper, all will start the wh***'s crying	wh***'s, wh***'s crying, peacekeeper	wh***'s, wh***'s crying	
HASOC (Marathi)	दादा साहेब फाळके नाव आहे मंद बुद्धी.	मंद, बुद्धी, आहे	मंद बुद्धी	0.67
English translation	Dadasaheb Phalke is the name, you slow-witted person.	slow-witted, is	slow-witted	
MACD (Hindi)	तेरी *ड में प्याज काट देगा गुज्जर **ड़ी के	*ड, **ड़ी, में	*ड, काट, **ड़ी	0.50
English translation	Gujjar will chop onions in your ass, you fu****	ass, fu****, in	ass, chop, fu****	
MACD (Tamil)	அப்ப ஓத்தா மாசமாக இருக்க முடியுமா உன்னால?	அப்ப, ஓத்தா, முடியுமா	ஓத்தா	0.50
English translation	Can you stay without fuc*** for a month?	then, fuc***, can you	fuc***	
MACD (Telugu)	వాడి ముందు పొద్దున మడ్డా శీమ్	మడ్డా, రేయ్	మడ్డా	0.50
English translation	Hey, show your dic* in front of him in the morning	dic*, Hey	dic*	
BD-SHS (Bengali)	শালা বাংলাদেশি মাদার**	মাদার**, শালা	মাদার**, শালা	0.67
English translation	Shala Bangladeshi mother*****	mother*****, Shala	mother*****, Shala	

efficiency and model accuracy. Other parameters, such as dropout rate and regularization strength, were tuned to prevent overfitting and ensure generalization across diverse samples. Each hyperparameter was systematically adjusted to achieve optimal performance within the constraints of our dataset and computational resources.

4.3 Results

In this section, we summarize all experiment results and analyses based on three tasks—(1) $\mathcal{M}_{\text{HSC}}$, (2) $\mathcal{M}_{\text{HII}}$, and (2) $\mathcal{M}_{\text{HIM}}$. For the $\mathcal{M}_{\text{HSC}}$ task, we compare two state-of-the-art transformer-based pre-trained models, i.e., multilingual and monolingual, on classification tasks over automatic metric-based evaluations. Regarding the $\mathcal{M}_{\text{HII}}$, we provide results generated by both IG and human evaluation. For the $\mathcal{M}_{\text{HIM}}$, we evaluate model performance solely based on human evaluation. Human annotations are often considered the standard for almost every NLP task. In numerous NLP applications, from machine translation to fact-checking, several past studies (Belz and Reiter 2006) have already depicted that automatic metric-based evaluations are insufficient. Although human-based evaluation requires much effort and significant evaluation time compared to automatic metric-based evaluations, a well-designed human evaluation over a small sample of model outputs attests to model accuracy for real-world applications. Due to these facts, we incorporate a human-based evaluation in our experiments.

4.3.1 SafeSpeech-HSC module

For the $\mathcal{M}_{\text{HSC}}$ task evaluation, four automatic metrics are used in our experiments: Accuracy (16), Precision (17), Recall (18), and F1 score (19). A macro-average version of these four metrics is considered as we observed class imbalance in our incorporated datasets.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (16)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (17)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (18)$$

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (19)$$

Table 5 shows the performance of two different transformer-based models: Google-MuRIL and l3cube-pune. For each model's output over every dataset, we compute all four metrics, along with dispersion (standard deviation) across cross-validation folds. Table 5 suggests that, except for Hindi, the l3cube-pune model is the top-scoring model for hate-detection tasks over all other datasets. This is possible as l3cube-pune pre-trained models are trained on monolingual data, i.e., language-specific data collected from several sources. 5 shows

4.3.2 SafeSpeech-HII module

For evaluating the $\mathcal{M}_{\text{HII}}$ module, we pick words with positive hate intensity from a given text and match those picked words based on the ground truth. The similarity between predicted hate words and ground truth hate words is calculated through the Jaccard Index (Jaccard 1912).

Human-based evaluation. For the hate words identification task of $\mathcal{M}_{\text{HII}}$ through human evaluation, we selected three graduate students from different departments (with dissimilar backgrounds) to ensure diversity among the human annotator pool. All annotations from these annotators are collected in a normal lab environment based on given instructions. Table 6 shows the top 3 words based on hate intensity and human evaluation for a single sample. We consider a sample of 40 texts from each dataset, from which we select the three top hate words per text identified by our proposed method. Each annotator is asked to determine three topmost hate words from each sample text. Finally, we obtain the Jaccard index similarity (Eq. 20) between model-predicted and human-detected hate words per dataset described in Table 6. The Jaccard index value indicates that the proposed method performs similarly to humans on the $\mathcal{M}_{\text{HSC}}$ test.

$$\text{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (20)$$

where A = model-predicted words and B = human-detected words.

Table 7 BERTScore as a text similarity metric between hate text and mitigated text

Dataset	BERTScore (avg.)
HASOC-Hindi	0.99
HASOC-Marathi	0.98
MACD-Hindi	0.98
MACD-Tamil	0.96
MACD-Telugu	0.96
BD-SHS-Bengali	0.99

Table 8 Input texts from different datasets, English translations for the input texts, the output of $\mathcal{M}_{HSC}$, $\mathcal{M}_{HII}$ and the best non-hate output generation through $\mathcal{M}_{HIM}$. The red colour indicates a high hate intensity

HASOC (Hindi)	
Input text	सब रं* नाच देखने मे व्यस्त जैसे ही कोई शांतीदूत के साथ कुछ होगा सब रं*रोना शुरू कर देंगे
English translation	All are busy watching the wh***'s dance as soon as something happens with the peacekeeper, all will start the wh***'s crying
$\mathcal{M}_{HSC}$	hate
$\mathcal{M}_{HII}$	सब रं* नाच देखने मे व्यस्त जैसे ही कोई शांतीदूत के साथ कुछ होगा सब रं*रोना शुरू कर देंगे
$\mathcal{M}_{HIM}$	सब लोग नाच देखने मे व्यस्त जैसे ही कोई शांतीदूत के साथ कुछ होगा सब नाच शुरू कर देंगे
English translation	All people are busy watching the dance as soon as something happens with the peacekeeper, all will start to dance
HASOC (Marathi)	
Input text	Text
Input text	दादा साहेब फाळके नाव आहे मंद बुद्धी.
English translation	Dadasaheb Phalke is the name, you slow-witted person.
$\mathcal{M}_{HSC}$	hate
$\mathcal{M}_{HII}$	दादा साहेब फाळके नाव आहे मंद बुद्धी .
$\mathcal{M}_{HIM}$	दादा साहेब फाळके नाव आहे पूर्ण बुद्धी.
English translation	Dadasaheb Phalke is the name, you full of wisdom.
MACD (Hindi)	
Input text	Text
Input text	भों** के इतना बड़ा धोखा मेरे साथ
English translation	Bhon*****, such a big betrayal to me
$\mathcal{M}_{HSC}$	hate
$\mathcal{M}_{HII}$	भों** के इतना बड़ा धोखा मेरे साथ
$\mathcal{M}_{HIM}$	नहीं भाई इतना बड़ा धोखा मेरे साथ
English translation	No brother, such a big betrayal to me
MACD (Tamil)	
Input text	Text
Input text	அப்ப ஓத்தா மாசமாக இருக்க முடியுமா உன்னால?
English translation	Can you stay without fuc*** for a month?
$\mathcal{M}_{HSC}$	hate
$\mathcal{M}_{HII}$	அப்ப ஓத்த மாசமாக இருக்க முடியுமா உன்னால
$\mathcal{M}_{HIM}$	சரினோ மாசமாக இருக்க முடியுமா உன்னால?
English translation	Can you be fine for a whole month?
MACD (Telugu)	
Input text	Text
Input text	వాడి ముందు పొద్దున ముడ్డా రేయ్
English translation	Hey, show your dic* in front of him in the morning
$\mathcal{M}_{HSC}$	hate
$\mathcal{M}_{HII}$	వాడి ముందు పొద్దున ముడ్డా రేయ్
$\mathcal{M}_{HIM}$	వాడి ముందు పొద్దున ప ర
English translation	Come in front of him in the morning
BD-SHS (Bengali)	
Input text	Text
Input text	শালা বাংলাদেশি মাদার**
English translation	Shala Bangladeshi mother*****
$\mathcal{M}_{HSC}$	hate
$\mathcal{M}_{HII}$	শালা বাংলাদেশি মাদার**
$\mathcal{M}_{HIM}$	অবশ্যই বাংলাদেশি একটা
English translation	Definitely a Bangladeshi one

4.3.3 SafeSpeech-HIM module

To mitigate hate, top κ words with positive hate intensity are replaced with the special token ‘[MASK]’. Using the MLM concept, we generated a set of candidates by replacing those masked hate words with

different high-probability words based on contexts. To select an optimal candidate for the candidate set, we use the BERTScore as a text similarity metric Table 7. Replacing hate words in a text with another highly probable word (yielding a modified text) doesn’t always reduce the degree of hate content. Incorporating a proper and faithful

Table 9 The original input text, top κ words (in red), and best candidate text iteration-wise. Examples are taken from BD-SHS (Bengali) and MACD (Telugu) datasets only

HASOC-Hindi	Text	Word-level hate intensity
Iteration 1		
Input text	सब रं* नाच देखने मे व्यस्त (English translation - All are busy watching the wh***s dance) — prediction is hate	
Top κ words	सब रं* नाच देखने मे व्यस्त	[0.0187, 0.9344, 0.1403, -0.0845, 0.0387, -0.0120]
Best candidate text	सब लोग नाच देखने मे व्यस्त (English translation - All people are busy watching the dance) — prediction is non-hate	[-0.0243, -0.1841, -0.1734, -0.0291, -0.0183, -0.0172]
HASOC-Marathi	Text	Word-level hate intensity
Iteration 1		
Input text	दादा साहेब फाळके नाव आहे मंद बुद्धी. (English translation - Dadasaheb Phalke is the name, you slow-witted person.) — prediction is hate	
Top κ words	दादा साहेब फाळके नाव आहे मंद बुद्धी.	[-0.3187, -0.0187, -0.2083, 0.0164, 0.1845, 0.9453, 0.6534]
Best candidate text	दादा साहेब फाळके नाव आहे पूर्ण बुद्धी. (English translation - Dadasaheb Phalke is the name, you full of wisdom.) — prediction is non-hate	[-0.1154, -0.2063, 0.1344, -0.1149, -0.2039, -0.1068, 0.1446]
BD-SHS-Bengali	Text	
Iteration 1		
Input text	শালা বাংলাদেশি মাদার** (English translation - Shala Bangladeshi mother*****) — prediction is hate	
Top κ words	শালা বাংলাদেশি মাদার**	[0.3723, 0.0958, 0.9231]
Best candidate text	শালা বাংলাদেশি একটা — prediction is hate	
Iteration 2		
Best candidate text from Iteration 1	শালা বাংলাদেশি একটা (English translation - Shala Bangladeshi one)	
Top κ words	শালা বাংলাদেশি একটা	[0.9489, -0.3140, -0.0277]
Best candidate text	অবশ্যই বাংলাদেশি একটা (English translation - Definitely a Bangladeshi one) — prediction is non-hate	[-0.1545, -0.9090, -0.3870]
MACD-Tamil	Text	
Iteration 1		
Input text	அப்ப ஓத்தா மாசமாக இருக்க முடியுமா உன்னால? (Can you stay without fuc*** for a month?) — prediction is hate	
Top κ words	அப்ப ஓத்தா மாசமாக இருக்க முடியுமா உன்னால?	[0.1077, 0.1153, -0.1715, -0.0814, 0.0317, -0.3854]
Best candidate text	சரினோ மாசமாக இருக்க முடியுமா உன்னால? (English translation - Can you be fine for a whole month?) — prediction is non-hate	[-0.1275, -0.1342, -0.1611, -0.1702, -0.1208, -0.1740]
MACD-Telugu	Text	
Iteration 1		
Input text	వాడి ముందు పొద్దున మడ్డా రేయ్ (Hey, show your dic* in front of him in the morning) — prediction is hate	
Top κ words	వాడి ముందు పొద్దున మడ్డా రేయ్	[-0.1324, -0.0012, -0.2451, 0.4534, 0.1354]
Best candidate text	వాడి ముందు పొద్దున ప రేయ్ — prediction is hate	
Iteration 2		
Best candidate text from Iteration 1	వాడి ముందు పొద్దున ప రేయ్ (English translation - Hey, come in front of him in the morning)	
Top κ words	వాడి ముందు పొద్దున ప రేయ్	[-0.2314, -0.1562, -0.3218, -0.2461, 0.3021]
Best candidate text	వాడి ముందు పొద్దున ప రా (English translation - Come in front of him in the morning) — prediction is non-hate	[-0.1241, -0.1212, -0.1341, -0.4244, -0.1214]

assessment is crucial to our proposed method. We performed a human-based evaluation for this hate intensity mitigation task with the same setup that we use for the hate words identification task of $\mathcal{M}_{\text{HII}}$; we gave a sample of 40 modified texts to those human annotators and asked them to rate them on a 1-5 Likert scale, where 5 denotes

the absolute mitigation of hate content from hate texts. We obtain average scores of 4.1 (out of 5), 4.2 (out of 5), and 3.8 (out of 5) corresponding to the three annotators w.r.t. the Likert scale. Statistically, all annotators have a $r = 0.86$ degree of agreement (Pearson correlation). High levels of annotator agreement and high average scores per

annotator indicate that our proposed model for mitigating hate speech performs well.

4.4 Case studies

Apart from quantitative analysis, Table 8 shows hate words identified by $\mathcal{M}_{\text{HII}}$ over a hate input and the new text generation by $\mathcal{M}_{\text{HIM}}$. Some inputs take a single iteration to be a non-hate text, and some input texts take two or more iterations. Table 9 shows such two examples in Telugu and Bengali, where the original text, top κ tokens (marked in red) for each iteration, and non-hate generation per iteration are given. We need to mention one example from Table 9 in Telugu i.e., “వాడి ముందు పొద్దున ప”, here, “క్రియ” in Telugu is an informal and colloquial expression used to call someone’s attention. The context is disrespectful; in certain situations, especially when used with someone unfamiliar or in a tense conversation, it can be perceived as disrespectful or dismissive.

5 Error analysis

Although our proposed system performs superiorly in $\mathcal{M}_{\text{HII}}$ and $\mathcal{M}_{\text{HIM}}$, it fails to produce legitimate outputs in some instances. We perform error analyses for two sub-tasks—i. $\mathcal{M}_{\text{HII}}$. ii. $\mathcal{M}_{\text{HIM}}$.

5.1 SafeSpeech-HII Module

Even though the proposed model shows promising results in $\mathcal{M}_{\text{HII}}$, we discover a limitation. The proposed model often fails to identify all hate words in a given text containing many hate words. In the following case, we have shown that several hate words remain even after we detect the text as hate in the given text.

- Given input: “बस एक ही शब्द है क*ला *ड की पैदाइश है मा***द इसकी माँ को मौलाना ने *क *क के इतना गन्दी औलाद पैदा किया है मा***द” (English translation - There is only one word, mother*****, the birth of a cut cock, mother***** fu**** his mother, such a dirty child of cock has been born, mother*****)
- Hate words (red colored): “बस एक ही शब्द है क*ला *ड की पैदाइश है मा***द इसकी माँ को मौलाना ने *क *क के इतना गन्दी औलाद पैदा किया है मा***द”
- Detected hate words (green colored): “बस एक ही शब्द है क*ला *ड की पैदाइश है मा***द इसकी माँ को मौलाना ने *क *क के इतना गन्दी औलाद पैदा किया है मा***द”

5.2 SafeSpeech-HIM module

In the $\mathcal{M}_{\text{HIM}}$, we find that the proposed model poses two types of errors.

- **Position of hate words in text:** Our proposed method’s performance in mitigating hate intensity significantly varies with hate word positions in a text. If a detected hate word appears at the beginning of a text, our proposed method often fails to mitigate hate intensity. For example, the text “सा* र* गोरखपुर को बदनाम करती” (English translation - Shali wh*** brings bad name to Gorkhpur) is reduced to “दलाल टीम गोरखपुर को बदनाम करती”. Red parts denote the hate content. So, we can observe that even after mitigating hate intensity, the resultant text still shows hate content.
- **Coagulation of multiple hate words in text:** Coagulation of multiple hate words in a text also deteriorates the proposed method’s credibility while reducing hate intensity content. Several observations on our proposed method outcomes show that if multiple hate words appear together in texts, our proposed method fails to mitigate hate intensity from those texts. For example, the texts “आपकी खु* *ली मिटाने के लिये हम *ल ल* *र तयार है” (English translation - We are ready to apply oil to soothe your itch) is reduced to “आपकी दर्द मिटाने के लिये हम *ड मारने को तयार है”. This demonstrates that the proposed method is ineffective in mitigating HS when multiple hate words appear together.
- **Substituting non-hate words with hate words in a texts:** In some examples the replacement of non-hate words (as mistakenly non-hate words are identified as hate) is by one hate word. So, here is an example wherein the first iteration system replaced the correct hate word. The example is “तेरी *ड में प्याज काट देगा गुज्जर भोस* के” (English translation - Gujjar will chop onions in your ass, you fu****) becomes “तेरी बिल में प्याज काट देगा गुज्जर भोस* के” but in the next iteration “तेरी बिल में प्याज काट देगा गुज्जर भोस* के” becomes “तेरी बिल में *ड काट देगा गुज्जर भोस* के”.

6 Conclusion and future work

In this paper, we introduce **SafeSpeech**, a novel three-module deep learning-based system designed to detect and mitigate the intensity of hate content in Indic languages. Both stages leverage state-of-the-art pre-trained language models, achieving competitive results against existing systems. To our knowledge, SafeSpeech is the first system specifically tailored for hate content mitigation in Indic languages. A notable feature of SafeSpeech is its integration of

Table 10 Combination of parameters for LSTM, BiLSTM, CNN models

Model name	Parameter name	Parameter value
LSTM	Epochs	10
	Batch size	32
	Embedding dimension	200, 300
	LSTM layer 1	128, <i>ReLU</i> activation
	LSTM layer 2	128, <i>ReLU</i> activation
	Dropout	0.1
	Dense layer 1	32, <i>ReLU</i> activation
	Dropout	0.1
	Dense layer 2	1, <i>sigmoid</i> activation
	Learning-rate	0.001
BiLSTM	Epochs	10
	Batch size	32
	Embedding dimension	200, 300
	BiLSTM layer 1	64
	BiLSTM layer 1	64
	Dense layer 1	256, <i>ReLU</i> activation
	Dense layer 2	1, <i>sigmoid</i> activation
	Learning-rate	0.001
CNN	Epochs	10
	Batch size	32
	Embedding dimension	200, 300
	Conv1D layer 1	128 filters, 3 kernel_size, <i>ReLU</i> activation, 1 stride
	Global MaxPooling1D 1	–
	Conv1D layer 2	256 filters, 4 kernel_size, <i>ReLU</i> activation, 1 stride
	Global MaxPooling1D 2	–
	Conv1D layer 3	512 filters, 5 kernel_size, <i>ReLU</i> activation, 1 stride
	Global MaxPooling1D 3	–
	Concatenate	Global MaxPooling1D 1, Global MaxPooling1D 2, Global MaxPooling1D 3
	Dense layer 1	256, <i>ReLU</i> activation
	Dense layer 2	1, <i>sigmoid</i> activation
	Learning-rate	0.001

self-explainable techniques and minimal reliance on labelled data, which significantly reduces the need for human annotations in hate speech detection compared to current models. Furthermore, the system operates without requiring domain expert involvement for the mitigation of hate content. Our extensive results section, which includes evaluations based on automatic metrics, underscores the strong performance of SafeSpeech. Additionally, comprehensive human evaluations affirm its reliability. To further demonstrate the effectiveness of our model, we present essential case studies with detailed steps for hate speech mitigation. In the error analysis section, we explore the limitations and boundaries of our system. Future work will focus on enhancing the proposed system's ability to process long-form texts and addressing the errors identified in this study.

Comparisons with state-of-the-art baselines

We used three modules: Hate Speech Classification, Hate Intensity Identification, and Hate Intensity Mitigation. For **Hate Speech Classification**, we compared the transformer-based BERT model with three non-transformer-based models: Long-Short-Term Memory (LSTM) (Hochreiter and Schmidhuber 1997), Bidirectional Long Short-Term Memory (BiLSTM) (Graves and Schmidhuber 2005), and Convolutional Neural Networks (CNNs) (Lecun et al. 1998). We found that the BERT model gives the most promising results compared to others. Table 10 shows a combination of parameters for LSTM, BiLSTM, CNN models.

Table 11 Precision, Recall, F1 score, and Accuracy of all the models (best model Table 5, LSTM, BiLSTM and CNN) applied on HASOC-Hindi, HASOC-Marathi, MACD-Hindi, MACD-Tamil, MACD-Telugu and BD-SHS-Bengali datasets

Dataset	Model	Accuracy	Precision	Recall	Macro F1
HASOC-Hindi	google/muril-base-cased	0.84	0.84	0.84	0.84
	LSTM	0.81	0.81	0.81	0.81
	BiLSTM	0.82	0.81	0.82	0.82
	CNN	0.82	0.82	0.82	0.82
HASOC-Marathi	l3cube-pune/marathi-bert	0.91	0.90	0.89	0.90
	LSTM	0.86	0.86	0.86	0.86
	BiLSTM	0.86	0.86	0.86	0.86
	CNN	0.87	0.86	0.87	0.87
MACD-Hindi	google/muril-base-cased	0.86	0.86	0.86	0.86
	LSTM	0.81	0.81	0.81	0.81
	BiLSTM	0.83	0.81	0.82	0.82
	CNN	0.82	0.82	0.82	0.82
MACD-Tamil	l3cube-pune/tamil-bert	0.88	0.88	0.88	0.88
	LSTM	0.78	0.78	0.78	0.78
	BiLSTM	0.79	0.78	0.79	0.78
	CNN	0.80	0.80	0.80	0.80
MACD-Telugu	l3cube-pune/telugu-bert	0.90	0.89	0.90	0.90
	LSTM	0.81	0.81	0.81	0.81
	BiLSTM	0.82	0.82	0.81	0.81
	CNN	0.83	0.82	0.83	0.83
BD-SHS-Bengali	l3cube-pune/bengali-bert	0.92	0.92	0.92	0.92
	LSTM	0.89	0.89	0.89	0.89
	BiLSTM	0.88	0.87	0.88	0.87
	CNN	0.88	0.88	0.88	0.88

- **LSTM:** This network is a recurrent neural network (RNN) architecture designed to model sequential data by effectively capturing long-term dependencies.
- **BiLSTM:** This network is an extension of LSTM networks, where data is processed in both forward and backward directions to capture patterns from past and future contexts.
- **CNN:** This is a type of deep learning model especially well-suited for processing data that has a grid-like topology, such as images, by leveraging spatial hierarchies in the data.

The architectural parameters used to train CNN, LSTM, and BiLSTM models are listed in Table 10, respectively. Table 11 shows the results in precision, Recall, F1 score, and Accuracy of all the models (best model Table (5), LSTM, BiLSTM and CNN) applied on HASOC-Hindi, HASOC-Marathi, MACD-Hindi, MACD-Tamil, MACD-Telugu and BD-SHS-Bengali datasets.

Existing methods for **Hate Intensity Identification** rely on annotated data, such as manual annotation of hate intensity for specific words, whereas our approach does not require such annotation. Since we don't have any direct work, we compare our approach to two popular explainable approaches: LIME (Local Interpretable Model-Agnostic Explanations) and IG (Integrated Gradient). We found that IG gives the better result. Table 12 shows the comparisons between LIME and IG.

Previous works on **Hate Intensity Mitigation** require supervised training on annotated datasets to identify hate spans and mitigate hate intensity. This framework's limitation is that it requires a hate-tagged corpus, which is expensive and complex to obtain. To bridge this gap, we leverage mask language modeling (MLM), which makes the system automatic, eliminating the need for annotated datasets. As our intensity mitigation approach is one of the first to incorporate automatic techniques through MLM, there are no directly comparable hate intensity mitigation models available.

Table 12 Comparisons of IG and LIME

Dataset	Sample text	IG	LIME
HASOC (Hindi)	सब रं* नाच देखने मे व्यस्त जैसे ही कोई शांतीदूत के साथ कुछ होगा सब रं*रोना शुरू कर देंगे	रं*, रं*रोना, शांतीदूत	रं*, शांतीदूत
English translation	All are busy watching the wh***s dance as soon as something happens with the peacekeeper, all will start the wh***s crying	wh***s, wh***s crying, peace-keeper	wh***s, peacekeeper
HASOC (Marathi)	दादा साहेब फाळके नाव आहे मंद बुद्धी.	मंद, बुद्धी, आहे	आहे, मंद
English translation	Dadasaheb Phalke is the name, you slow-witted person.	slow-witted, is	you, slow
MACD (Hindi)	तेरी *ड में प्याज काट देगा गुज्जर **डी के	*ड, **डी, में	*ड, काट, देगा
English translation	Gujjar will chop onions in your ass, you fu****	ass, fu****, in	ass, chop, will
MACD (Tamil)	அப்ப ஓத்தா மாசுமாக இருக்க முடியுமா உன்னால?	ஓத்தா, ஓத்தா, முடி-யுமா	ஓத்தா
English translation	Then, can you stay without fuc*** for a month?	then, fuc***, can you	fuc***
MACD (Telugu)	వాడి ముందు పొద్దున పడ్డా రేమ్	పడ్డా, రేమ్	పడ్డా, ముందు
English translation	Hey, show your dic* in front of him in the morning	dic*, Hey	dic*, in front
BD-SHS (Bengali)	শালা বাংলাদেশি মাদার**	মাদার**, শালা	শালা, বাংলাদেশি
English translation	Shala Bangladeshi mother*****	mother*****, Shala	Shala, Bangladeshi

Acknowledgements This research is partially supported by the Indo-French Centre for the Promotion of Advanced Research (IFCPAR/CEFIPRA) through CSRP Project No. 6702-2.

Author contributions Ms. Ghosh: Literature review, conceptualization of the three-module model that detects hate text and hate words and mitigates them, coding the model, experimentation, manuscript preparation; Mr. Singh: Literature review, the addition of extra annotation, coding the system, experimentation, manuscript preparation; Mr. Mahapatra: Conceptualized the hate mitigation method, numerous technical help for implementing deep models, help in preparing the manuscript; Mr. Saha: Normalization and interpretation of attribution scores; Dr. Senapati: enriching the research ideas and manuscript revision; Prof. Garain: Problem formulation, supervision of entire research, validating experimental results, manuscript checking.

Data availability The data used in this research is public-domain dataset. The additional annotation added by us for human evaluation and the entire codebase will be made available under a GitHub link after the paper is accepted.

Declarations

Conflict of interest No conflict of interest.

Ethical and informed consent Not applicable for the dataset used, which is available in open domain.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and

your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- Agarap AF (2018) Deep learning using rectified linear units (relu). CoRR arXiv:1803.08375
- Albadi N, Kurdi M, Mishra S (2019) Investigating the effect of combining gru neural networks with handcrafted features for religious hatred detection on arabic twitter space. Soc Netw Anal Min 9(1):41. <https://doi.org/10.1007/s13278-019-0587-5>
- Anand M, Eswari R (2019) Classification of abusive comments in social media using deep learning. In: 2019 3rd international conference on computing methodologies and communication (ICCMC), pp 974–977. <https://doi.org/10.1109/ICCMC.2019.8819734>
- Badjatiya P, Gupta S, Gupta M, Varma V (2017) Deep learning for hate speech detection in tweets. In: Proceedings of the 26th international conference on world wide web companion. WWW '17 Companion, pp. 759–760. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE. <https://doi.org/10.1145/3041021.3054223>
- Bahdanau D, Cho K, Bengio Y (2014) Neural machine translation by jointly learning to align and translate. <https://doi.org/10.48550/arXiv.1409.0473>
- Bashar MA, Nayak R (2020) Qutnocturnal@hasoc'19: CNN for hate speech and offensive content identification in hindi language. CoRR arXiv:2008.12448
- Belz A, Reiter E (2006) Comparing automatic and human evaluation of NLG systems. In: 11th Conference of the European chapter of the association for computational linguistics, pp. 313–320
- Bhardwaj M, Akhtar M.S, Ekbal A, Das A, Chakraborty T (2020) Hostility detection dataset in hindi. CoRR arXiv:2011.03588

- Binder A, Montavon G, Bach S, Müller K, Samek W (2016) Layer-wise relevance propagation for neural networks with local renormalization layers. CoRR [arXiv:1604.00825](https://arxiv.org/abs/1604.00825)
- Bloomberg (2019) Twitter, Facebook join global pledge to fight hate speech online. Accessed 13 Aug 2024
- Burnap P, Williams ML (2014) Hate speech, machine classification and statistical modelling of information flows on twitter: Interpretation and communication for policy decision making
- Burnap P, Williams ML (2016) Us and them: identifying cyber hate on twitter across multiple protected characteristics. EPJ Data Sci 5:1–15
- Cercas Curry A, Abercrombie G, Rieser V (2021) ConvAbuse: data, analysis, and benchmarks for nuanced abuse detection in conversational AI. In: Proceedings of the 2021 conference on empirical methods in natural language processing. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, pp 7388–7403. <https://doi.org/10.18653/v1/2021.emnlp-main.587>
- Chen Y (2011) Detecting offensive language in social medias for protection of adolescent online safety
- Clarke C, Hall M, Mittal G, Yu Y, Sajeev S, Mars J, Chen M (2023) Rule by example: Harnessing logical rules for explainable hate speech detection. In: Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers). Association for Computational Linguistics, Toronto, Canada, pp 364–376. <https://doi.org/10.18653/v1/2023.acl-long.22>
- Cortes C, Vapnik V (1995) Support-vector networks. Mach Learn 20:273–297
- Dahiya S, Sharma S, Sahnan D, Goel V, Chouzenoux E, Elvira V, Majumdar A, Bandhakavi A, Chakraborty T (2021) Would your tweet invoke hate on the fly? forecasting hate intensity of reply threads on twitter. In: Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining. KDD '21. Association for Computing Machinery, New York, NY, USA, pp 2732–2742. <https://doi.org/10.1145/3447548.3467150>
- Das AK, Asif AA, Paul A, Hossain MN (2021) Bangla hate speech detection on social media using attention-based recurrent neural network. J Intell Syst 30(1):578–591. <https://doi.org/10.1515/jisys-2020-0060>
- Davidson T, Warmsley D, Macy M (2017) Weber I Automated hate speech detection and the problem of offensive language. In: Proceedings of the International AAAI Conference on Web and Social Media 11(1):512–515. <https://doi.org/10.1609/icwsm.v11i1.14955>
- Davidson T, Warmsley D, Macy MW, Weber I (2017) Automated hate speech detection and the problem of offensive language. CoRR [arXiv:1703.04009](https://arxiv.org/abs/1703.04009)
- Devlin J, Chang M, Lee K, Toutanova K (2018) BERT: pre-training of deep bidirectional transformers for language understanding. CoRR [arXiv:1810.04805](https://arxiv.org/abs/1810.04805)
- Dinakar K, Reichart R (2021) Lieberman H Modeling the detection of textual cyberbullying. In: Proceedings of the International AAAI Conference on Web and Social Media 5(3):11–17. <https://doi.org/10.1609/icwsm.v5i3.14209>
- Djuric N, Zhou J, Morris R, Grbovic M, Radosavljevic V, Bhamidipati N (2015) Hate speech detection with comment embeddings. In: Proceedings of the 24th international conference on world wide web. WWW '15 Companion. Association for Computing Machinery, New York, NY, USA, pp 29–30. <https://doi.org/10.1145/2740908.2742760>
- Djuric N, Zhou J, Morris R, Grbovic M, Radosavljevic V, Bhamidipati N (2015) Hate speech detection with comment embeddings. In: Proceedings of the 24th international conference on world wide web. WWW '15 Companion. Association for Computing Machinery, New York, NY, USA, pp 29–30. <https://doi.org/10.1145/2740908.2742760>
- Fischer A, Halperin E, Canetti D, Jasini A (2018) Why we hate. Emot Rev 10(4):309–320. <https://doi.org/10.1177/1754073917751229>
- Ghosh K, Senapati A, Narzary M, Brahma M (2023) Hate speech detection in low-resource bodo and assamese texts with ml-dl and bert models. Scalable Comput Pract Exp 24(4):941–955
- Ghosh K, Senapati DA (2022) Hate speech detection: a comparison of mono and multilingual transformer model with cross-language evaluation. In: Dita S, Trillanes A, Lucas RI (eds) Proceedings of the 36th Pacific Asia conference on language, information and computation. Association for Computational Linguistics, Manila, Philippines, pp 853–865. <https://aclanthology.org/2022.paclic-1.94>
- Ghosh K, Senapati A (2024) Hate speech detection in low-resourced indian languages: an analysis of transformer-based monolingual and multilingual models with cross-lingual experiments. Natural Language Processing
- Ghosh K, Sonowal D, Basumatary A, Gogoi B, Senapati A (2023) Transformer-based hate speech detection in assamese. In: 2023 IEEE Guwahati Subsection Conference (GCON), pp 1–5. <https://doi.org/10.1109/GCON58516.2023.10183497>
- Graves A, Schmidhuber J (2005) Framewise phoneme classification with bidirectional LSTM and other neural network architectures. Neural Netw 18(5):602–610. <https://doi.org/10.1016/j.neunet.2005.06.042>. (IJCNN 2005)
- Greevy E, Smeaton AF (2004) Classifying racist texts using a support vector machine. In: Proceedings of the 27th annual international ACM SIGIR conference on research and development in information retrieval. SIGIR '04. Association for Computing Machinery, New York, NY, USA, pp 468–469. <https://doi.org/10.1145/1008992.1009074>
- Greevy E, Smeaton AF (2004) Classifying racist texts using a support vector machine. In: Proceedings of the 27th annual international ACM SIGIR conference on research and development in information retrieval. SIGIR '04. Association for Computing Machinery, New York, NY, USA, pp 468–469. <https://doi.org/10.1145/1008992.1009074>
- Gupta V, Roychowdhury S, Das M, Banerjee S, Saha P, Mathew B, Vanchinathan H, Mukherjee A (2024) Macd: multilingual abusive comment detection at scale for indic languages. In: Proceedings of the 36th international conference on neural information processing systems. NIPS '22. Curran Associates Inc., Red Hook, NY, USA
- Guterres A et al (2019) United nations strategy and plan of action on hate speech. Taken from: [https://www.un.org/en/genocideprevention/documents/U](https://www.un.org/en/genocideprevention/documents/U%20Strategy) (20Strategy)
- Hallinan S, Liu A, Choi Y, Sap M (2023) Detoxifying text with MaRCO: Controllable revision with experts and anti-experts. In: Rogers A, Boyd-Graber J, Okazaki N (eds) Proceedings of the 61st annual meeting of the association for computational linguistics (volume 2: short papers). Association for Computational Linguistics, Toronto, Canada, pp 228–242. <https://doi.org/10.18653/v1/2023.acl-short.21>
- Hase P, Chen C, Li O, Rudin C (2019) Interpretable image recognition with hierarchical prototypes. In: Proceedings of the AAAI conference on human computation and crowdsourcing, vol 7, pp 32–40
- Hemphill L Hate speech in social media: How platforms can do better. <https://news.umich.edu/hate-speech-in-social-media-how-platforms-can-do-better/> Accessed 30 Dec 2023
- Hochreiter S, Schmidhuber J (1997) Long short-term memory. Neural Comput 9(8):1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735> (<https://direct.mit.edu/neco/article-pdf/9/8/1735/813796/neco.1997.9.8.1735.pdf>)
- Hutchinson A (2019) Instagram adds new anti-bullying measures, including comment warnings and user restrictions. Accessed 30 Aug 2024
- Hutchinson A (2020) Twitter is testing prompts which would recommend users hide potentially offensive replies. Accessed 30 Aug 2024
- Jaccard P (1912) The distribution of the flora in the alpine zone. 1. New phytol 11(2):37–50
- Joshi R (2022) L3cube-hindbert and devbert: Pre-trained bert transformer models for devanagari based hindi and marathi languages. arXiv preprint [arXiv:2211.11418](https://arxiv.org/abs/2211.11418)

- Joshi R (2022) L3cube-mahacorporus and mahabert: Marathi monolingual corpus, marathi bert language models, and resources. In: Proceedings of the WILDRE-6 workshop within the 13th language resources and evaluation conference, pp. 97–101. European Language Resources Association, Marseille, France
- Kemp S (2022) Digital 2022: Global overview report. <https://datareport.al.com/reports/digital-2022-global-overview-report>. Accessed on 14 June 2024
- Kemp S (2024) Digital 2024: Global overview report. <https://datareport.al.com/reports/digital-2024-global-overview-report>. Accessed 14 June 2024
- Kennedy CJ, Bacon G, Sahn A, Vacano C (2020) Constructing interval variables via faceted rasch measurement and multitask deep learning: a hate speech application. CoRR [arXiv:2009.10277](https://arxiv.org/abs/2009.10277)
- Khanuja S, Bansal D, Mehtani S, Khosla S, Dey A, Gopalan B, Margam DK, Aggarwal P, Nagipogu RT, Dave S, Gupta S, Gali SCB, Subramanian V, Talukdar PP (2021) Muril: Multilingual representations for indian languages. CoRR [arXiv:2103.10730](https://arxiv.org/abs/2103.10730)
- Kwok I (2013) Wang Y Locate the hate: Detecting tweets against blacks. In: Proceedings of the AAAI Conference on Artificial Intelligence 27(1):1621–1622. <https://doi.org/10.1609/aaai.v27i1.8539>
- Laub Z (2019) Hate speech on social media: Global comparisons. Council on foreign relations 7
- Lecun Y, Bottou L, Bengio Y, Haffner P (1998) Gradient-based learning applied to document recognition. *Proceeding IEEE* 86(11):2278–2324. <https://doi.org/10.1109/5.726791>
- Li O, Liu H, Chen C, Rudin C (2018) Deep learning for case-based reasoning through prototypes: A neural network that explains its predictions. In: Proceedings of the AAAI conference on artificial intelligence, vol 32
- Liu S, Forss T (2014) Combining n-gram based similarity analysis with sentiment analysis in web content classification. In: Proceedings of the international joint conference on knowledge discovery, knowledge engineering and knowledge management—volume 1. IC3K 2014. SCITEPRESS—Science and Technology Publications, Lda, Setubal, PRT, pp 530–537. <https://doi.org/10.5220/0005170305300537>
- Lundberg S.M, Lee S (2017) A unified approach to interpreting model predictions. CoRR [arXiv:1705.07874](https://arxiv.org/abs/1705.07874)
- Madsen A, Reddy S, Chandar S (2022) Post-hoc interpretability for neural NLP: a survey. *ACM Comput Surv* <https://doi.org/10.1145/3546577>
- Maity K, Poornash AS, Bhattacharya S, Phosit S, Kongsamlit S, Saha S, Pasupa K (2024) Hatethaisent: sentiment-aided hate speech detection in thai language. *IEEE Trans Comput Soc Syst*. <https://doi.org/10.1109/TCSS.2024.3376958>
- Mandl T, Modha S, Majumder P, Patel D, Dave M, Mandlia C, Patel A (2019) Overview of the hasoc track at fire 2019: Hate speech and offensive content identification in indo-european languages. In: Proceedings of the 11th forum for information retrieval evaluation. FIRE '19. Association for Computing Machinery, New York, NY, USA, pp 14–17. <https://doi.org/10.1145/3368567.3368584>
- Mandl T, Modha S, Kumar MA, Chakravarthi BR (2020) Overview of the hasoc track at fire 2020: Hate speech and offensive language identification in tamil, malayalam, hindi, english and german. In: Forum for information retrieval evaluation. FIRE 2020. Association for Computing Machinery, New York, NY, USA, pp 29–32. <https://doi.org/10.1145/3441501.3441517>
- Masud S, Bedi M, Khan MA, Akhtar MS, Chakraborty T (2022) Proactively reducing the hate intensity of online posts via hate speech normalization. <https://doi.org/10.48550/ARXIV.2206.04007>
- Mathew B, Saha P, Yimam SM, Biemann C, Goyal P, Mukherjee A (2020) Hatexplain: a benchmark dataset for explainable hate speech detection. CoRR [arXiv:2012.10289](https://arxiv.org/abs/2012.10289)
- Mehdad Y, Tetreault J (2016) Do characters abuse more than words? In: Proceedings of the 17th annual meeting of the special interest group on discourse and dialogue, pp 299–303
- Meng Q, Suresh T, Lee RK-W, Chakraborty T (2023) Predicting hate intensity of twitter conversation threads. *Knowledge-Based Syst* 275:110644 <https://doi.org/10.1016/j.knosys.2023.110644>
- Modha S, Mandl T, Shahi GK, Madhu H, Satapara S, Ranasinghe T, Zampieri M (2021) Overview of the hasoc subtrack at fire 2021: Hate speech and offensive content identification in english and indo-aryan languages and conversational hate speech. In: Forum for information retrieval evaluation. FIRE 2021. Association for Computing Machinery, New York, NY, USA, pp 1–3. <https://doi.org/10.1145/3503162.3503176>
- Nagar S, Barbhuiya FA (2023) Dey K Towards more robust hate speech detection: using social context and user data. *Soc Netw Anal Min* 13(1):47. <https://doi.org/10.1007/s13278-023-01051-6>
- Nations U (2023) Understanding hate speech. United Nations
- Nguyen A, Martínez MR (2019) Mononet: Towards interpretable models by learning monotonic features. CoRR [arXiv:1909.13611](https://arxiv.org/abs/1909.13611)
- Nobata C, Tetreault J, Thomas A, Mehdad Y, Chang Y (2016) Abusive language detection in online user content. In: Proceedings of the 25th international conference on world wide web. WWW '16. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, pp 145–153. <https://doi.org/10.1145/2872427.2883062>
- Noh H, Hong S, Han B (2015) Learning deconvolution network for semantic segmentation. CoRR [arXiv:1505.04366](https://arxiv.org/abs/1505.04366)
- ohchr: Moderating online content: fighting harm or silencing dissent? <https://www.ohchr.org/en/stories/2021/07/moderating-online-content-fighting-harm-or-silencing-dissent>. Accessed 30 Dec 2023
- O'Regan C Hate speech regulation on social media: An intractable contemporary challenge. <https://researchoutreach.org/articles/hate-speech-regulation-social-media-intractable-contemporary-challenge/> Accessed 30 Dec 2023
- Paul S, Saha S, Singh JP (2023) Covid-19 and cyberbullying: deep ensemble model to identify cyberbullying from code-switched languages during the pandemic. *Multimed Tools Appl* 82:8773–8789. <https://doi.org/10.1007/s11042-021-11601-9>
- Raj R, Srivastava S, Saumya S (2020) Nsit & iitdwd @ hasoc 2020: Deep learning model for hate-speech identification in indo-european languages. In: FIRE
- Reportal D (2021) Digital 2021: Global overview report. Tomado de: <https://datareportal.com/reports/digital-2021-global-overview-report>. Accessed on 14 June 2024
- Ribeiro MT, Singh S, Guestrin C (2016) “why should I trust you?”: explaining the predictions of any classifier. CoRR [arXiv:1602.04938](https://arxiv.org/abs/1602.04938)
- Romim N, Ahmed M, Talukder H, Islam MS (2020) Hate speech detection in the bengali language: a dataset and its baseline evaluation. CoRR [arXiv:2012.09686](https://arxiv.org/abs/2012.09686)
- Romim N, Ahmed M, Islam M.S, Sen Sharma A, Talukder H, Amin MR (2022) BD-SHS: A benchmark dataset for learning to detect online bangla hate speech in different social contexts. In: Proceedings of the thirteenth language resources and evaluation conference. European Language Resources Association, Marseille, France, pp 5153–5162. <https://aclanthology.org/2022.lrec-1.552>
- Saksesi AS, Nasrun M, Setianingsih C (2018) Analysis text of hate speech detection using recurrent neural network. In: 2018 international conference on control, electronics, renewable energy and communications (ICCEREC), pp 242–248. <https://doi.org/10.1109/ICCEREC.2018.8712104>
- Schmidt A, Wiegand M (2017) A survey on hate speech detection using natural language processing. In: Proceedings of the Fifth International Workshop on Natural Language Processing for Social

- Media. Association for Computational Linguistics, Valencia, Spain, pp 1–10. <https://doi.org/10.18653/v1/W17-1101>
- Sennrich R, Haddow B, Birch A (2015) Neural machine translation of rare words with subword units. CoRR [arXiv:1508.07909](https://arxiv.org/abs/1508.07909)
- Sharma A, Kabra A, Jain M (2022) Ceasing hate with moh: Hate speech detection in hindi-english code-switched language. Inf Process Manag 59(1):102760. <https://doi.org/10.1016/j.ipm.2021.102760>
- Shrikumar A, Greenside P, Kundaje A (2017) Learning important features through propagating activation differences. CoRR [arXiv:1704.02685](https://arxiv.org/abs/1704.02685)
- Singh A, Sharma D, Singh VK (2025) Misogynistic attitude detection in youtube comments and replies: a high-quality dataset and algorithmic models. Comput Speech Lang 89:101682. <https://doi.org/10.1016/j.csl.2024.101682>
- Springenberg JT, Dosovitskiy A, Brox T, Riedmiller M (2014) Striving for simplicity: the all convolutional net. arXiv preprint [arXiv:1412.6806](https://arxiv.org/abs/1412.6806)
- Sundararajan M, Taly A, Yan Q (2017) Axiomatic attribution for deep networks. CoRR [arXiv:1703.01365](https://arxiv.org/abs/1703.01365)
- Times (2019) Facebook says it's removing more hate speech than ever before. But there's a catch. Accessed 13 Aug 2024
- Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I (2017) Attention is all you need. CoRR [arXiv:1706.03762](https://arxiv.org/abs/1706.03762)
- Vidgen B, Thrush T, Waseem Z, Kiela D (2021) Learning from the worst: dynamically generated datasets to improve online hate detection. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th international joint conference on natural language processing (volume 1: long papers). Association for Computational Linguistics, Online, pp 1667–1682. <https://doi.org/10.18653/v1/2021.acl-long.132>
- voxp0l: Equality european union's rights and citizenship programme. 2014. beyond the “big three”—alternative platforms for online hate speech. <https://www.voxpol.eu/download/report/Beyond-the-Big-Three-Alternative-platforms-for-online-hate-speech.pdf>. Accessed 30 Dec 2023
- Wang W, Zhao H, Zhuang H, Shah N, Padman R (2020) Dycrs: dynamic interpretable postoperative complication risk scoring. In: Proceedings of the web conference 2020. WWW '20, pp. 1839–1850. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3366423.3380253>
- Waseem Z, Hovy D (2016) Hateful symbols or hateful people? predictive features for hate speech detection on twitter. In: Proceedings of the NAACL student research workshop, pp 88–93
- Waseem Z, Hovy D (2016) Hateful symbols or hateful people? Predictive features for hate speech detection on Twitter. In: Andreas J, Choi E, Lazaridou A (eds) Proceedings of the NAACL Student Research Workshop. Association for Computational Linguistics, San Diego, California, pp 88–93. <https://doi.org/10.18653/v1/N16-1013>
- Wiegand M, Ruppenhofer J, Schmidt A, Greenberg C (2018) Inducing a lexicon of abusive words—a feature-based approach. In: Walker M, Ji H, Stent A (eds) Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers). Association for Computational Linguistics, New Orleans, Louisiana, pp 1046–1056. <https://doi.org/10.18653/v1/N18-1095>
- Wolf T, Debut L, Sanh V, Chaumond J, Delangue C, Moi A, Cistac P, Rault T, Louf R, Funtowicz M, et al (2020) Transformers: State-of-the-art natural language processing. In: EMNLP (system Demo Track)
- Yang H, Rudin C, Seltzer MI (2016) Scalable bayesian rule lists. CoRR [arXiv:1602.08610](https://arxiv.org/abs/1602.08610)
- Yaraghi N How should social media platforms combat misinformation and hate speech? <https://www.brookings.edu/articles/how-should-social-media-platforms-combat-misinformation-and-hate-speech/> Accessed 30 Dec 2023
- Zhang T, Kishore V, Wu F, Weinberger KQ, Artzi Y (2019) Bertscore: Evaluating text generation with BERT. CoRR [arXiv:1904.09675](https://arxiv.org/abs/1904.09675)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.